

A Toolkit for Succinct Lattice-Based Zero Knowledge Proofs

Beatrice Biasioli^{1,2}, Madalina Bolboceanu^{1,2}, Vadim Lyubashevsky¹, Antonio Merino-Gallardo^{1,2},
Michał Osadnik³, Gregor Seiler¹, and Patrick Steuer^{1,4}

¹ IBM Research Europe, Zurich, Switzerland

beatrice.biasioli@ibm.com, madalina.bolboceanu@ibm.com, vad@zurich.ibm.com, antonio@m-g.es,
gseiler@posteo.net, ick@zurich.ibm.com

² University of Potsdam, Potsdam, Germany

³ Aalto University, Espoo, Finland michal.osadnik@aalto.fi

⁴ Radboud University, Nijmegen, Netherlands

Abstract. The development of proof systems whose security relies on the hardness of lattice problems has been a fruitful research area in recent years. By leveraging the techniques introduced in LaBRADOR (Beullens, Seiler, Crypto 2023), the state-of-the-art lattice-based schemes have very fast provers and have output sizes under 100KB for arbitrarily large statements. These proofs are in fact the smallest, and often have the fastest provers, out of all post-quantum schemes.

In addition to succinctness, many applications also require witness privacy. Achieving this can, in theory, be done by combining LaBRADOR with a linear-size zero-knowledge proof. While such a combination has already been described in the LaBRADOR paper itself, as well as in the works of Albrecht et al. (Eurocrypt 2024) and del Pino et al. (Crypto 2025), its concrete costs remained unexplored.

In this work, we provide the first concrete construction and implementation that adds zero-knowledge proofs to LaBRADOR by integrating the linear-size zero-knowledge proof from (Lyubashevsky, Nguyen, Plançon, Crypto 2022) into the protocol. We describe the non-trivial challenges that this entails and show practicality of the construction by benchmarking several use-cases. We make the proof system and primitives accessible by extending the LaZer library (Lyubashevsky, Seiler, Steuer, CCS 2024) in a way that they can easily be used in other applications.

Keywords: Lattice Cryptography, Zero-Knowledge, Succinct Proofs, Implementation, Privacy, Quantum-Safe

1 Introduction

Succinct lattice-based proof systems have been shown to be competitive with proof systems based on other hardness assumptions. In recent times, there has been a lot of progress, and many concretely efficient lattice-based constructions have been proposed, e.g. [11, 13, 15, 16, 32, 42].

The two other main categories of succinct proof systems are pairing-based SNARKs, e.g. [21, 28, 31], and hash-based STARKs, e.g. [8, 9, 46]. SNARKs offer extremely small constant-size proofs but are not secure against quantum computers and have relatively slow provers. STARKs are quantum safe, do not need any structured computational assumptions, and offer faster runtimes, but produce quite large proofs. In comparison, lattice-based proof systems have the potential to combine quantum-safety and fast runtimes with proof sizes that are smaller than those of the STARKs.

Indeed, the computationally friendly mathematical structure of lattice commitments that are simple high-dimensional linear functions over polynomial rings has been shown to allow for lattice-based polynomial commitment schemes that have leading prover runtimes [32, 42] and competitive verifier runtimes in the single-digit millisecond range [32]. Polynomial commitment schemes (PCSs) are the core cryptographic component in modern proof system constructions. At the same time, the proof sizes of these lattice-based PCSs are leading among quantum-safe PCSs.

An important but somewhat neglected feature in current succinct lattice-based proof systems and their implementations is the zero-knowledge property. Only the linear-size LNP proof system [36] and its implementation in the LaZer library [40] comes with zero-knowledge.

For many proof system applications, the succinctness property is the main enabling feature. A typical example is signature aggregation, where a large number of signatures are compressed into a small proof that can be verified in very little time. Here, the signatures do not need to be hidden as there is no privacy requirement, and hence the proof system does not need to have the zero-knowledge property.

The situation is completely different in privacy-preserving applications where zero-knowledge is the central feature provided by the proof system. Typical application examples are blind signature schemes and anonymous credential systems constructed with the help of zero-knowledge proofs. In such protocols, users possess signatures from some underlying basic signature scheme, but they do not want to reveal these signatures in the clear, as this would make them traceable. Instead, the users only provide zero-knowledge proofs of possession of the signatures and hence keep the signatures private.

In light of the ongoing and pressing transition to quantum-safe cryptography, it is also important to transition advanced privacy-preserving cryptographic protocols such as Privacy Pass, and hence, there is a strong need for practical quantum-safe zero-knowledge proof systems and implementations thereof. Furthermore, given that the main quantum-safe signature scheme Dilithium [25], which is standardized as ML-DSA by NIST, is a lattice scheme, it is often necessary to prove statements about lattice schemes. In this case there are significant synergies when the proof system is also based on lattices. In particular, the proof system then shares a lot of the mathematical structure with the scheme to-be-proven, and hence can be implemented using similar implementation techniques. Furthermore, a large fraction of the operations in the scheme can be phrased as polynomial relations that are native to the proof system. Thus, the proof statement can often be constructed manually, and one does not need to resort to circuit representations and (trusted) compiler tool chains.

This provides further reasons to consider lattice-based zero-knowledge proofs beyond small proof sizes and computational performance. In the present paper we propose a lattice-based proof system that offers both succinct proof sizes as well as zero-knowledge. Concretely, our proof system is a combination of a variant of the **LaBRADOR** proof system with a modification of the **LNP** proof system that we call **LNP-Lite**. While such constructions have been suggested before, we provide the first detailed instantiation and the first implementation of such a proof system.

In more detail, prior works [3, 44] have suggested constructing a lattice-based zero-knowledge proof system with succinct proof sizes by proving the input relation with **LNP** in a first step. In **LNP** proofs, only the last prover message (the so-called masked opening $\mathbf{z}$) scales linearly with the witness size. The remaining inner prover messages are sublinear in size, so they can be revealed in the clear. This allows the verifier to compute all Fiat-Shamir hashes. Then, what remains is to prove knowledge of $\mathbf{z}$ fulfilling all verifier checks performed on it in **LNP**. These are all simple polynomial relations of the type supported by **LaBRADOR**. So instead of also outputting the masked opening, we can prove knowledge of it using **LaBRADOR**, resulting in a succinctly-sized zero-knowledge proof.

There are two drawbacks to this approach. First, as also realized by [44], the inner **LNP** prover messages, while sublinear in size, are still concretely quite large, and hence blow up the proof size of the combined proof system. Second, and more importantly, the masking of the secret in $\mathbf{z}$ blows up the size of the witness for **LaBRADOR**, and hence results in poor runtimes. Therefore, a better approach is to first run several iterations of a sublinear-size proof protocol that reduces the witness size. This was already suggested in [11] but without a concrete instantiation. The sublinear-size protocol does not need to be fully zero-knowledge. It suffices if the inner prover messages, which are output, are simulatable. On the contrary, the last prover messages are not output and do not need to be simulatable. They are proven by the next iteration of the protocol, or, in case of the last protocol iteration, proven in zero-knowledge by **LNP**. Finally, one can proceed as before and prove the last prover message of **LNP** with further iterations of **LaBRADOR** where no zero-knowledge requirements exist anymore. In this way, the masking in **LNP** only increases the size of a witness that is much smaller than the initial input witness.

For the sublinear-size proof protocol, we use the improved variant of **LaBRADOR** from [13], which we refer to as **LaBRADOS**. In this protocol all the inner prover messages are wrapped in binding lattice commitments to minimize their size. These commitments can be made simulatable easily by adding sufficient randomness so that the commitments become hiding under **LWE**. Our **LNP** variant **LNP-Lite** contains commitment

wrappers similarly to [44] in order to reduce the impact on the final proof size. Moreover, we further modify LNP in several ways. Most importantly, we propose a new technique to mask the random projection that allows us to instantiate the protocol with a modulus q small enough for the LaBRADOR implementation. The naive way of masking and committing to the random projection requires moduli that are too big for LaBRADOR. Finally, we implement LNP-Lite on top of the same arithmetic implementation that is used by the LaBRADOR implementation in order to obtain a unified implementation of the complete proof system. Also, we added support for the quadratic input constraints required for proving LaBRADOS. Those constraints are not supported by the LaZer implementation of LNP.

We showcase the performance and practicality of our construction and implementation by using it for proving several useful lattice statements that require sublinear proof sizes as well as zero-knowledge. Concretely, our benchmarks center around proving symmetric-key primitives such as collision-resistant hash functions and PRG evaluation. Such proofs are useful in the construction of numerous high-level protocols. For example, proving PRG evaluation can be used for constructing a verifiable encryption scheme.

2 Preliminaries

Notation Let q be an odd prime. We denote by $\mathbb{Z}_q$ the ring of integers mod q . We write $[n] = \{1, 2, \dots, n\}$. We say $x \leftarrow S$ when $x \in S$ is sampled uniformly at random from a finite set S and similarly $x \leftarrow \chi$ when x is sampled according to a distribution χ . We denote by $\mathcal{U}(S)$ the uniform distribution over the finite set S . Given two moduli $q \geq p \geq 2$, we define the rounding function from $\mathbb{Z}_q$ to $\mathbb{Z}_p$ that maps $x \in \mathbb{Z}_q$ to $\lceil x \rceil_{q \rightarrow p} := \lceil \frac{p}{q} \cdot x \rceil \bmod p$ and we extend this function to integer vectors coordinate-wise. Let d be a power of two, and let $\mathcal{R}, \mathcal{R}_q$ be the rings $\mathbb{Z}[X]/(X^d + 1)$ and $\mathbb{Z}_q[X]/(X^d + 1)$, for any integer q . Most of the times, q is chosen such that $X^d + 1$ splits in two irreducible factors mod q . We work with $d = 256$ or 512 . For a polynomial $f = a_0 + a_1X + \dots + a_{d-1}X^{d-1} \in \mathcal{R}$, we denote by $\text{ct}(f)$ its constant-term, i.e. $\text{ct}(f) = a_0$. We use $\sigma_{-1}(\cdot)$ to denote the “conjugation automorphism”, i.e. $\sigma_{-1} \in \text{Aut}(\mathcal{R})$ such that $\sigma_{-1}(X) = X^{-1}$. Vectors over $\mathbb{Z}$, $\mathcal{R}$, $\mathcal{R}_q$ are denoted by italic-bold lower-case and matrices by italic-bold capital letters. For any polynomial vector $\mathbf{a} \in \mathcal{R}^n$, we denote by $\dim(\cdot)$ their dimension as integer vector, i.e. $\dim(\mathbf{a}) = nd$. We denote by $\mathbf{1}_r$ the column vector containing all entries equal to (the constant polynomial) 1, and by $\mathbf{0}_r$ the one containing all entries equal to (the constant polynomial) 0. If $\mathbf{a} \in \mathcal{R}_q^{n_a}$ and $\mathbf{b} \in \mathcal{R}_q^{n_b}$ are vectors, we denote by $(\mathbf{a}, \mathbf{b}) \in \mathcal{R}_q^{n_a+n_b}$ their concatenation. A function $f : \mathbb{N} \rightarrow \mathbb{R}$ is said to be negligible if for all $c \in \mathbb{N}$ there exists some $n_c \in \mathbb{N}$ such that for all $n > n_c$, $|f(n)| < \frac{1}{n^c}$. In the context of probabilities, we say $a \approx b$ if $a - b$ is negligible. Further notation and additional preliminaries can be found in Section A.

2.1 The Challenge Space

The challenge space $\mathcal{C} \subseteq \mathcal{R}_q$ is chosen such that $c - c'$ is invertible for any pair of distinct $c, c' \in \mathcal{C}$ and with bounded ℓ_2 norm and *operator norm*, i.e. $\|c\| \leq \tau$ and $\|c\|_{\text{op}} \leq T$, for any $c \in \mathcal{C}$, for some constants $\tau, T \in \mathbb{R}$, where $\|c\|_{\text{op}} := \sup_{r \in \mathcal{R} \setminus \{0\}} \frac{\|cr\|}{\|r\|}$. A popular choice [11] is to sample c with small $\|c\| = \tau$ by taking a fixed number of short non-zero entries (e.g. $\pm 1, \pm 2$) that ensures a large-enough size of $\mathcal{C}$, while the invertibility is guaranteed by their shortness as proved in [39, Cor.1.2].

2.2 Lattice Assumptions

Here we recall the computational lattice problems Module-Short Integer Solution (M-SIS), Module-Learning With Errors (M-LWE), and Module-Learning with Rounding (M-LWR) [25, 34].

Definition 1 (M-SIS). *For integers $M, N, q > 0$ and real number $\beta > 0$, we define $M\text{-SIS}_{q,M,N,\beta}$ problem as follows: Given $\mathbf{A} \leftarrow \mathcal{R}_q^{M \times N}$, find $\mathbf{v} \in \mathcal{R}_q^N$ such that $\mathbf{A}\mathbf{v} = \mathbf{0}$ and $0 < \|\mathbf{v}\| \leq \beta$.*

Definition 2 (M-SIS with ℓ_∞ norm). *We define $M\text{-SIS}_{q,M,N,\beta}^\infty$ analogously to Definition 1 with $\|\cdot\|$ replaced with $\|\cdot\|_\infty$.*

Definition 3 (Decision M-LWE). For integers $n, m, q > 0$ and χ a distribution over $\mathcal{R}$, we define the decision $M\text{-LWE}_{q,m,n,\chi}$ problem as follows: for $\mathbf{s} \leftarrow \chi^n$, distinguish between the following cases

- $(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$, for $\mathbf{A} \leftarrow \mathcal{R}_q^{m \times n}$, $\mathbf{e} \leftarrow \chi^m$,
- $(\mathbf{A}, \mathbf{b})$, for $\mathbf{A} \leftarrow \mathcal{R}_q^{m \times n}$, $\mathbf{b} \leftarrow \mathcal{R}_q^m$.

We recall the Extended M-LWE problem [38], used in proving zero-knowledge of our proofs of knowledge.

Definition 4 (Extended M-LWE). For integers $n, m, q > 0$, χ a distribution over $\mathcal{R}$, a positive real number $\sigma > 0$ and a challenge space $\mathcal{C} \subseteq \mathcal{R}_q$, we define the Extended $M\text{-LWE}_{q,m,n,\chi}$ problem as follows: for $\mathbf{r} \leftarrow \chi^n$, distinguish between the following two cases

- $(\mathbf{B}, \mathbf{B}\mathbf{r}, c, \mathbf{z}, \text{sign}(\langle \mathbf{z}, \mathbf{c}\mathbf{r} \rangle))$, for $\mathbf{B} \leftarrow \mathcal{R}_q^{m \times n}$, $\mathbf{z} \leftarrow D_\sigma^n$, $c \leftarrow \mathcal{C}$,
- $(\mathbf{B}, \mathbf{u}, c, \mathbf{z}, \text{sign}(\langle \mathbf{z}, \mathbf{c}\mathbf{r} \rangle))$, for $\mathbf{B} \leftarrow \mathcal{R}_q^{m \times n}$, $\mathbf{u} \leftarrow \mathcal{R}_q^m$, $\mathbf{z} \leftarrow D_\sigma^n$, $c \leftarrow \mathcal{C}$,

where $\text{sign}(a) = 1$, if $a \geq 0$, and 0 otherwise.

The Module-Learning with Rounding problem deals with two integer moduli, $q \geq p \geq 2$.

Definition 5 (Decision M-LWR). For integers $n', m', q \geq p > 0$ and χ be a distribution over $\mathcal{R}$, then the decision $M\text{-LWR}_{q,p,m',n',\chi}$ problem as follows: for $\mathbf{s} \leftarrow \chi^{n'}$, distinguish between the following two cases:

- $(\mathbf{A}, \lceil \mathbf{A}\mathbf{s} \rceil_{q \rightarrow p})$, for $\mathbf{A} \leftarrow \mathcal{R}_q^{m' \times n'}$,
- $(\mathbf{A}, \mathbf{u})$, for $\mathbf{A} \leftarrow \mathcal{R}_q^{m' \times n'}$, $\mathbf{u} \leftarrow \mathcal{R}_p^{m'}$.

2.3 LaBRADOR

LaBRADOR is a proof system introduced by Beullens and Seiler [11] whose security is based on the hardness of the M-SIS problem. This proof system enjoys a concretely small communication size, and is tailored for proving lattice-related relations. LaBRADOR has a core base protocol and becomes *succinct* by leveraging recursion: the last message of one iteration of the base protocol becomes the witness for the next one, and the new statement corresponds to the verifier's checks from the previous iteration. In this way, the size of the witness is reduced with each iteration, until no further progress is made, and the prover finally sends the last witness for the verifier to check directly.

More concretely, the LaBRADOR proof system proves knowledge of short witness vectors $\mathbf{s}_1, \dots, \mathbf{s}_r \in \mathcal{R}^n$, satisfying a *global* norm bound $\sum_{i \in [r]} \|\mathbf{s}_i\|^2 \leq \beta$ and dot-product constraints $f : \mathcal{R}_q^n \times \dots \times \mathcal{R}_q^n \rightarrow \mathcal{R}_q$ of the form

$$f(\mathbf{s}_1, \dots, \mathbf{s}_r) \triangleq \sum_{i,j \in [r]} a_{ij} \langle \mathbf{s}_i, \mathbf{s}_j \rangle + \sum_{i \in [r]} \langle \varphi_i, \mathbf{s}_i \rangle - b, \quad (1)$$

where $a_{ij}, b \in \mathcal{R}_q$, $\varphi_i \in \mathcal{R}_q^n$, such that $a_{ij} = a_{ji}$. For some constraints f' , we are only interested in the constant term $\text{ct}(\cdot)$ of their output, so the other coefficients of b are not included in the statement. We collect these *constant term* constraints in a family $\mathcal{F}'$, separated from the *full* constraints, that we collect in a family $\mathcal{F}$. It is worth noting that the witness vectors will actually be proven to satisfy *looser* norm bounds, by a slack factor of $c_{\text{Lab}} := \sqrt{128/30}$, due to the use of random projections.

This proof system is not *zero-knowledge*, i.e., the prover messages leak information about the witness.

Commitment Layers. LaBRADOR works with two layers of commitments, that their authors refer to as *inner* and *outer* layers. The inner commitments are Ajtai commitments of the witness vectors $\mathbf{s}_i$, of rank k_{in} , whereas the outer ones are Ajtai commitments, of rank k_{mid} , of the decompositions of the inner commitments. In anticipation of the additional layer of commitments that we consider in the next section, we refer to the outer commitment layer of LaBRADOR instead as *middle* commitments.

LaBRADOS. A recent work [13] introduced a variant of LaBRADOR that further compresses the size of the prover's messages sent in each iteration of the protocol (see [13, Sec.7.3]), and that serves as an important building block in our construction. We refer to this variant of LaBRADOR as LaBRADOS.

LaBRADOS works by binary-decomposing the original messages of LaBRADOR, and sending *outer* commitments of them. This means that the decompositions need to be included in the last prover message (i.e. the witness for the next iteration) and the verifier has to check that they are indeed binary and the correct openings to the outer commitments. One of the main advantages of this outer layer is that the rank of these commitments can be very small by relying on the hardness of M-SIS for infinity norm bound 1 (following the analysis in Dilithium, see [25, Appendix C.3]).

Another advantage is that, since the projections are no longer sent in the clear, one can prove individual norm bounds on the witness vectors, rather than a global norm bound. The downside is that, rather than checking the exact ℓ_2 -norm of the projections, the verifier only checks that they have been decomposed in binary with a certain number of bits. This leads to a slack factor of $c_{\text{Lab-new}} := 9.75/0.74$, by the use of Theorem 4.

Lastly, the main reason why we use LaBRADOS is that everything the prover sends (except for the last message) are Ajtai commitments. These can be easily made *hiding* by appending a long enough uniform binary vector before committing and relying on the hardness of the M-LWE problem.

2.4 LNP

Lyubashevsky, Nguyen, and Plançon introduced in [36] a proof system for lattice-related relations that achieves zero-knowledge, and that we refer to as LNP. Unlike LaBRADOR, the proof size of LNP is *linear*, since the size of the last prover message scales linearly with the size of the witness. Its security also relies on the assumed hardness of the M-SIS problem.

3 A Succinct Zero-Knowledge Proof System

We obtain a succinct zero-knowledge proof system by combining iterations of different protocols in the following way. We start by proving knowledge of the input witness with LaBRADOS equipped with hiding commitments. The last message of LaBRADOS, i.e., the output witness, is then recursively fed as input to the same protocol, and several iterations are run until the size of the witness is sufficiently small. We then proceed to feed the resulting small witness to LNP-Lite, a compressed version of the linear-size protocol LNP that is tailored to the output of LaBRADOS and that we describe in Section 3.3. The output of LNP-Lite could safely be made public, but we further compress it by running several iterations of LaBRADOS (now with non-hiding commitments), and finishing with several iterations of the original LaBRADOR. Once the size of the final witness cannot be decreased any more, the witness is finally sent to the verifier. We give an overview of this succinct zero-knowledge proof in Figure 1.

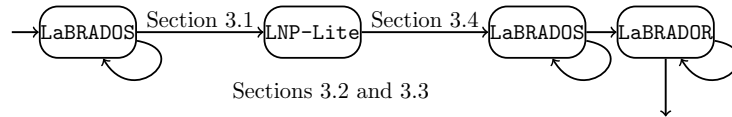


Fig. 1. Overview of the succinct zero-knowledge proof system from Section 3.

3.1 Understanding the Output of LaBRADOS

The variant of LNP that we describe in the subsequent sections is tailored to proving the output of LaBRADOS. In this section, we take a closer look at the latter, with an emphasis on what the output witness looks like and what the output statement to prove is.

LaBRADOS works with the polynomial ring $\mathcal{R}$ of degree $d = 256$ and the challenge space $\mathcal{C}$ (Section 2.1). The input of the LaBRADOS iterations is made of short witness vectors $\mathbf{s}_1, \dots, \mathbf{s}_r \in \mathcal{R}^n$, satisfying norm bounds $\|\mathbf{s}_i\| \leq \beta_i$ and constant-term and full-domain dot-product constraints of type (1).

Output Witness. As in LaBRADOR, the last message of LaBRADOS contains a linear combination of witness vectors, $\mathbf{z} := \sum_{i \in [r]} c_i \mathbf{s}_i$, with random challenges $c_i \leftarrow \mathcal{C}$ from the verifier. To ensure that the size of the coefficients of $\mathbf{z}$ does not blow up when recursing this protocol, [11] considers decomposing $\mathbf{z}$ in two parts, $\mathbf{z}_0$ and $\mathbf{z}_1$, with respect to a basis b as $\mathbf{z} = \mathbf{z}_0 + b\mathbf{z}_1$. Additionally, the output of LaBRADOS contains the openings of the middle commitments, collected in $\mathbf{x}_{\text{mid}}$, and the openings to the outer commitments and the random binary vectors used to make the outer commitments hiding, all binary vectors collected in $\mathbf{x}_{\text{bin}}$. Overall, the output witness is $(\mathbf{z}_0, \mathbf{z}_1, \mathbf{x}_{\text{mid}}, \mathbf{x}_{\text{bin}}, \sigma_{-1}(\mathbf{x}_{\text{bin}}))$, where the presence of the last element is justified in next subsection.

Output Statement. In order to ensure the binding property of the commitments, all the output witness vectors need to satisfy certain ℓ_2 -norm bounds, which are derived from the bases used to decompose their elements. Additionally, the output vectors satisfy constant-term and full dot-product constraints of type (1). They include the final aggregated dot-product constraint, the correctness of all commitment layers, and of the so-called *garbage terms* needed for checking the final aggregated constraint. Finally, we need to perform a binary check for $\mathbf{x}_{\text{bin}}$. We follow the standard technique of including the additional vector $\sigma_{-1}(\mathbf{x}_{\text{bin}})$ in the witness, and proving its well-formedness together with the appropriate quadratic equation (see Section A.3 for more detail). Note that for this vector we do not provide an approximate norm proof, as its norm bound trivially follows from that of $\mathbf{x}_{\text{bin}}$. Overall, the only quadratic checks are the correctness of the quadratic garbage terms, which contains the dot-products $\langle \mathbf{z}_i, \mathbf{z}_j \rangle$ for $i, j \in \{0, 1\}$, and the binary equation, which contains the term $\langle \mathbf{x}_{\text{bin}}, \sigma_{-1}(\mathbf{x}_{\text{bin}}) \rangle$.

Soundness. The proof of knowledge soundness follows, via Lemma 4, the same lines as the proof of [11, Thm 5.1] and assumes the binding of the outer commitment layer. The only difference is that, as LaBRADOS proves individual projections of the witness vectors as binary vectors using $\log(9.75\beta_i)$ bits (Theorem 3), it leads to proving approximate norm bounds with a slack factor $c_{\text{Lab-new}} := 9.75/0.74$, by Theorem 4. The statement of knowledge soundness is as follows.

Theorem 1. *Let $\mathcal{C} \subset \mathcal{R}_q$ be the challenge space from Section 2.1, of polynomials with operator norm bounded by T . Suppose M -SIS is hard for ℓ_2 norm bound 2γ and rank k_{mid} , and for ℓ_2 norm bound $\max\{8T\gamma', 2\gamma' + 4T \cdot \frac{9.75}{0.74} \cdot \max \beta_i\}$, rank k_{in} , and $\frac{9.75}{0.74}\beta_i < \frac{q}{91}$. Then LaBRADOS is a knowledge-sound proof with knowledge soundness $\kappa := 1/2^{128} + (1/q)^\lambda + 1/q^{d/2} + 2/|\mathcal{C}|$ and norm slack $c_{\text{Lab-new}} := \frac{9.75}{0.74}$, i.e. the extractor is only guaranteed to extract witness vectors $\mathbf{s}_i$ with norm bounds $\|\mathbf{s}_i\| \leq \frac{9.75}{0.74}\beta_i$.*

3.2 Overview of Improvements to LNP

The linear-size zero-knowledge protocol LNP [36] lies at the heart of our construction. The iterations of LaBRADOS run at the beginning ensure that the input to LNP is small. Similarly, the iterations of LaBRADOS and LaBRADOR run on the output witness of LNP ensure that the final witness sent in the end to the verifier will be succinct. This section deals with reducing the communication of the LNP protocol itself. We describe the main challenges we overcome and techniques we employ for obtaining a compressed version of LNP. The resulting protocol we propose is LNP-Lite, and we provide a full description in Section 3.3. We note that since this protocol is meant to be run on the output of LaBRADOS, we make some optimizations tailored to it.

New Challenge Space and Witness. The challenge space $\mathcal{C}$ that we use for this protocol is the same as LaBRADOS (Section 2.1), not stable under the automorphism σ_{-1} , as opposed to LNP [36]. It is thus crucial to include $\sigma_{-1}(\mathbf{x}_{\text{bin}})$ in the witness so as to prove that $\mathbf{x}_{\text{bin}}$ is binary.

Additional Commitment Layer. We compress LNP by adding an outer commitment layer to all but the last prover message, in a similar way that LaBRADOS is a compressed version of LaBRADOR. The original LNP commitments are then called *inner* commitments, which in our case are just hiding Ajtai commitments. The outer ones are also Ajtai commitments, but these do not need to be hiding. It is easy to check that extending LNP with outer commitments preserves its zero-knowledge (see Lemma 5).

Increase of the Projection Slack. Proving binary with LNP would become impractical if one simply extends [36] with outer commitments. The problem lies in the fact that the masked projections, designed to obtain approximate ℓ_2 norms on the input witness vectors, are no longer sent directly to the verifier, but only a commitment to their binary decompositions. In particular, the masked projection of the output binary vector $\mathbf{x}_{\text{bin}} \in \mathcal{R}^n$ follows, due to rejection sampling, a distribution close to D_σ , for $\sigma := \gamma\sqrt{337\sqrt{nd}}$, so we could write it using $\log(14\sigma)$ bits. Suppose that the prover proves that the decomposed projection is indeed binary and written in that many bits. Then, adapting the Johnson-Lindenstrauss proof to deal with the ℓ_∞ norm of the masked projections (Theorem 5), we could prove that $\|\mathbf{x}_{\text{bin}}\| \leq \frac{2 \cdot 14\sigma}{0.67}$ with high probability, leading to a slack factor $\alpha := 2 \cdot \frac{14}{0.67} \cdot \gamma\sqrt{337} \approx 12275$. To prove that $\mathbf{x}_{\text{bin}}$ contains, for example $n = 1024$ binary polynomials, we would need to set $2\|\mathbf{x}_{\text{bin}}\|^2 \leq 2\alpha^2 nd \leq q$, to ensure that the binary equation does not wrap around mod q (see Section A.3), leading to a modulus $q \approx 2^{46.2}$ for $d = 256$. For comparison, [36] can provide an approximate proof for $\mathbf{x}_{\text{bin}}$ with a slack factor $\alpha := 2\sqrt{256/26} \cdot \sqrt{2} \cdot \gamma\sqrt{337} \approx 2606.8$, ([36, Proposition 5.1]), for a repetition constant γ set, e.g., equal to 16, which leads to setting a modulus $q \approx 2^{41.7}$, for the same n .

Two-Step Projection. We employ a technique that allows us to obtain a smaller slack than the naive approach outlined above. In fact, we get a smaller slack than the original LNP. In our two-step projection approach, we delay using the expensive Gaussian masking and instead prove binary for vectors of *small fixed* dimension, regardless of the dimension of the input binary vectors, allowing for a smaller modulus q . Thus, the standard deviation of the Gaussian masking becomes a fixed parameter, independent of the input norm bounds. More precisely, we could write the projection of $\mathbf{x}_{\text{bin}}$, a vector in $\mathbb{Z}^{256}$, on $k+1$ bits, as $k := \lceil \log(9.75\sqrt{nd}) \rceil$ (Theorem 3). In fact, it could be written with high probability on just k bits, so one could think of the high-order bit as an extra bit equal to 0. Then we could mask the lower k bits of the projection with a mask $\mathbf{u}$ uniformly sampled over $[-2^{k-1}, 2^{k-1} - 1]^{256}$ and reject if there are more than $N = 32$ nonzero high-order values (called *carries*), being -1 or 1, across the 256 high-order values (in $\{-1, 0, 1\}$) of each of the projection entries. One could safely send the low-order bits of the projection, collected as $\mathbf{v}_0 \in \mathbb{Z}^{256}$, as they are uniform, but not the high-order values, collected as $\mathbf{v}_1 \in \{-1, 0, 1\}^{256}$, since they leak possible carries. This is where the second projection comes into play: we prove that this vector $\mathbf{v}_1$ is ternary by writing it as $\mathbf{v}_1 := \mathbf{w} - \mathbf{t}$ for $\mathbf{w}, \mathbf{t} \in \{0, 1\}^{256}$ and by providing an approximate proof for $(\mathbf{w}, \mathbf{t})$ as in [36], whose norm is exactly the number of carries, less than N . Its projection needs to be masked for assuring zero-knowledge and could be sent in the clear, without computing its outer commitment. Note that, compared to $\mathbf{x}_{\text{bin}}$, this vector has only $n = 2$ binary ring elements. Using the tighter bound $\sqrt{N} = \sqrt{32}$ for $\|(\mathbf{w}, \mathbf{t})\|$ and applying the rationale of [36, Proposition 5.1], one needs to set a modulus $q \approx 2^{28.7}$ to prove $(\mathbf{w}, \mathbf{t})$ is binary. Projecting individually each of the four LaBRADOS output vectors leads to proving binary for a vector of $n = 8$ binary ring elements, which increases q to being $\approx 2^{30.7}$. Note that the infinity norm of the projection of our vector $\mathbf{x}_{\text{bin}}$ is at most $\|\mathbf{u}\|_\infty + \|2^k \mathbf{v}_1\|_\infty + \|\mathbf{v}_0\|_\infty \leq 2^{k-1} + 2^k + 2^{k-1} = 2^{k+1} \approx 2 \cdot 9.75\sqrt{nd}$, which, applying Theorem 4, leads to $\|\mathbf{x}_{\text{bin}}\| \leq 2^{\frac{9.75}{0.74}}\sqrt{nd}$, giving us a slack factor $\alpha := 2 \cdot \frac{9.75}{0.74} \approx 26.35$, doubled compared to the one of LaBRADOS. Therefore, using the same requirement on q to prove $\mathbf{x}_{\text{bin}}$ is binary leads us to a modulus $q \approx 2^{28.4}$.

No Bimodal Rejection Sampling. The *bimodal* rejection sampling [24] used in approximate proofs in [36] involves a bit that needs to be considered as part of the witness, and thus makes the projection constraints quadratic in the witness. In our case, we instead opted for the standard rejection sampling.

3.3 LNP-Lite

We present the linear-size protocol that effectively provides zero-knowledge to our whole construction. It is based on the LNP [36] protocol, upgraded with the modifications described in Section 3.2. Its setup consists of the same polynomial ring $\mathcal{R}$ of degree $d = 256$, modulus q as LaBRADOS, and challenge space $\mathcal{C}$ (Section 2.1), for easier composition of the protocols. This protocol proves knowledge of the LaBRADOS output vectors, denoted for simplicity by $\mathbf{s}_i \in \mathcal{R}^{n_i}$, $i \in [5]$, with bounded norms β_i , and satisfying the verification checks of LaBRADOS. Notice that in these checks, the vector $\mathbf{s}_3$ appears only in the linear part. In addition, the dimensions n_i could be different and, in fact, we do not require them to be the same, as in LaBRADOS. We

proceed to describe the protocol. We apply the outer commitment layer as mentioned in Section 2.3. Hence, unless otherwise specified, all the prover's messages are decomposed in binary and the prover sends instead an outer commitment of them. For simplicity of exposition, we do not make this outer layer explicit in the description.

Commit. To ensure zero-knowledge, we sample additional masks whose binary decompositions are collected in a new witness vector $\mathbf{s}_6 \in \mathcal{R}^{n_6}$:

- $\mathbf{u}_i \leftarrow [-2^{k_i-1}, 2^{k_i-1} - 1]^{256}$, $i \leq 4$ masks the projections of $\mathbf{s}_i$, for $k_i := \lceil \log(9.75\beta_i) \rceil$. We identify each $\mathbf{u}_i$ with the corresponding polynomial $u_i \in \mathcal{R}$,
- $y \leftarrow D_\sigma$ over $\mathcal{R}$ masks the projection of high-order values in $\{-1, 0, 1\}$,
- $g_i \leftarrow \{x \in \mathcal{R}_q : \text{ct}(x) = 0\}$, $i \leq \lceil 128/\log q \rceil$ masks the aggregation of constant-term constraints.

The prover computes and sends the hiding commitment of $\tilde{\mathbf{s}} := (\mathbf{s}_1, \dots, \mathbf{s}_6)$. For this, he samples a random binary vector $\mathbf{r}_s \in \mathcal{R}^{n_s}$, to make it indistinguishable from random under M-LWE:

$$\mathbf{t}_s := \mathbf{A}_1 \tilde{\mathbf{s}} + \mathbf{A}'_1 \mathbf{r}_s \in \mathcal{R}_q^{k_s}, \quad (2)$$

where $\mathbf{A}_1 \in \mathcal{R}_q^{k_s \times \sum_i n_i}$, $\mathbf{A}'_1 \in \mathcal{R}_q^{k_s \times n_s}$ are public matrices.

Project in Two Steps. In the first step, the prover replaces the norm statements of the vectors $\mathbf{s}_i$, $i \in [4]$, with modular Johnson-Lindenstrauss projections [11, 13, 30, 36] of their coefficient vectors $\mathbf{s}_i \in \mathbb{Z}^{256n_i}$. Upon receiving $\mathbf{\Pi}_i \leftarrow \text{Bin}_1^{256 \times 256n_i}$ from the verifier, the prover computes the projections $\mathbf{\Pi}_i \mathbf{s}_i \in \mathbb{Z}_q^{256}$. Note that the prover does not need to compute the projections of $\mathbf{s}_5$ or of $\mathbf{s}_6$. We can assume that the maps $\mathbf{\Pi}_i$ come from the same projection matrix, by properly removing some columns to compute the respective projections. As $k_i = \lceil \log(9.75\beta_i) \rceil$, $i \in [4]$, we get $\|\mathbf{\Pi}_i \mathbf{s}_i\|_\infty \leq 2^{k_i}$ with high probability. We mask this projection using $\mathbf{u}_i \leftarrow [-2^{k_i-1}, 2^{k_i-1} - 1]^{256}$, and decompose it as

$$\mathbf{\Pi}_i \mathbf{s}_i + \mathbf{u}_i = 2^{k_i} \mathbf{v}_i^{(1)} + \mathbf{v}_i^{(0)} \in \mathbb{Z}_q^{256},$$

with $\mathbf{v}_i^{(0)} \in [-2^{k_i-1}, 2^{k_i-1} - 1]^{256}$, $\mathbf{v}_i^{(1)} \in \{-1, 0, 1\}^{256}$. The role of $\mathbf{u}_i$ here is to uniformize the low-order values, $\mathbf{v}_i^{(0)}$, of the projection $\mathbf{\Pi}_i \mathbf{s}_i$. Sending them as ring elements $v_i^{(0)} \in \mathcal{R}$ to the verifier would then not leak any information. The high-order values, $\mathbf{v}_i^{(1)}$, would leak possible carries that took place when padding, so they need to be committed. We also control the number of carries, i.e., the number of nonzero entries that the ternary vector $\mathbf{v}_i^{(1)}$ has, which is why we reject whenever $\|\mathbf{v}_i^{(1)}\|^2 > N$, for a fixed threshold N . To prove that $\mathbf{v}_i^{(1)}$ is ternary, we write it as $\mathbf{v}_i^{(1)} := \mathbf{w}_i - \mathbf{t}_i$, for $\mathbf{w}_i, \mathbf{t}_i \in \{0, 1\}^{256}$, such that $\mathbf{w}_i$ encodes the 1's of $\mathbf{v}_i^{(1)}$ and $\mathbf{t}_i$ encodes the -1's of $\mathbf{v}_i^{(1)}$, i.e., for any $j \in [256]$,

$$\mathbf{w}_i[j] = \begin{cases} 1 & \text{if } \mathbf{v}_i^{(1)}[j] = 1 \\ 0 & \text{else} \end{cases}, \quad \mathbf{t}_i[j] = \begin{cases} 1 & \text{if } \mathbf{v}_i^{(1)}[j] = -1 \\ 0 & \text{else} \end{cases}.$$

Therefore, $\|\mathbf{v}_i^{(1)}\|^2 = \|(\mathbf{w}_i, \mathbf{t}_i)\|^2 \leq N$. As $d = 256$, each binary vector $\mathbf{w}_i$ or $\mathbf{t}_i$ corresponds to a binary ring element w_i or t_i . Thus, from the four projections, we collect them all in $\tilde{\mathbf{v}}_1 \in \mathcal{R}^8$. As the challenge space is not invariant under σ_{-1} , we also need to commit to $\tilde{\mathbf{v}}_2 := \sigma_{-1}(\tilde{\mathbf{v}}_1) \in \mathcal{R}^8$. Note that $\|\tilde{\mathbf{v}}_1\|^2 = \|\tilde{\mathbf{v}}_2\|^2 \leq 4N$.

The prover computes and sends the hiding commitment of $\tilde{\mathbf{v}} := (\tilde{\mathbf{v}}_1, \tilde{\mathbf{v}}_2) \in \mathcal{R}^{16}$. For this, he samples a random binary vector $\mathbf{r}_v \in \mathcal{R}^{n_v}$, to make it indistinguishable from random under M-LWE:

$$\mathbf{t}_v := \mathbf{A}_2 \tilde{\mathbf{v}} + \mathbf{A}'_2 \mathbf{r}_v \in \mathcal{R}_q^{k_v}, \quad (3)$$

where $\mathbf{A}_2 \in \mathcal{R}_q^{k_v \times 16}$, $\mathbf{A}'_2 \in \mathcal{R}_q^{k_v \times n_v}$ are public matrices. Along with $\mathbf{t}_v$, the prover also sends the low-order bits, $v_i^{(0)}$, $i \in [4]$.

Starting from the second step of projection, we follow the lines of [36], for proving knowledge of two witness vectors, $\tilde{\mathbf{s}}$ and $\tilde{\mathbf{v}}$ satisfying dot-product constraints and norm bounds. Thus, the prover creates a masked projection $\mathbf{z} := \tilde{\mathbf{\Pi}} \tilde{\mathbf{v}}_1 + \mathbf{y} \in \mathbb{Z}_q^{256}$ of only $\tilde{\mathbf{v}}_1 \in \mathcal{R}^8$, using a projection map $\tilde{\mathbf{\Pi}} \leftarrow \text{Bin}_1^{256 \times 2048}$ received

from the verifier, and a Gaussian mask $\mathbf{y} \leftarrow D_\sigma$ from $\mathbf{s}_6$ for zero-knowledge. Note that $\tilde{\mathbf{v}}_2 = \sigma_{-1}(\tilde{\mathbf{v}}_1)$ does not need to be projected, as its shortness trivially follows from proving its well-formedness. The prover applies rejection sampling (Lemma 3 (1)) and sends $\mathbf{z}$, upon acceptance. Then, the verifier checks if $\|\mathbf{z}\| \leq \sigma\sqrt{2 \cdot 256}$. By rejection sampling, $\mathbf{z}$ follows a distribution close to D_σ , so this check holds with high probability. For this particular step, we do not apply the outer commitment, as here $\mathbf{z}$ is just one polynomial. In addition, the norm check will not be part of the input statement of the upcoming LaBRADOS iteration.

Aggregate the Constraints. In this step, the prover aggregates the constraints, following standard techniques [11, 13, 36]. First, the constant-term constraints, the ones from the LaBRADOS checks and additional ones, are aggregated with random integer challenges from the verifier. The additional constant-term constraints are needed for proving the correctness of:

- the masked projections of $\mathbf{s}_i$, $i \leq 4$, all linear in $\tilde{\mathbf{s}}, \tilde{\mathbf{v}}$,
- $\tilde{\mathbf{v}}_1$ being binary, containing the dot-product $\langle \tilde{\mathbf{v}}_1, \tilde{\mathbf{v}}_2 \rangle$,
- $\tilde{\mathbf{v}}_2 = \sigma_{-1}(\tilde{\mathbf{v}}_1) \in \mathcal{R}^8$, all linear in $\tilde{\mathbf{v}}$,
- the projection of $\tilde{\mathbf{v}}_1$, all linear in $\tilde{\mathbf{v}}$.

Aggregating these constraints preserves the zero constant coefficient. This is performed $\lambda := \lceil 128/\log q \rceil$ many times, as the soundness of one is just $1/q$. As these aggregations, called h_i , for $i \in [\lambda]$, could leak information on the witness vectors when being sent, the prover masks them with the g_i 's from $\mathbf{s}_6$. By the definition of g_i , the coefficients of h_i are all uniform, except the constant coefficient, which is 0. The verifier can then check if $\text{ct}(h_i) = 0$.

For proving well-formedness of the h_i 's, the prover constructs new quadratic constraints in $\tilde{\mathbf{s}}, \tilde{\mathbf{v}}$ and aggregates them, along with the full constraints from LaBRADOS checks, with random polynomial challenges from the verifier. The prover then obtains a single constraint, quadratic in $\tilde{\mathbf{s}}, \tilde{\mathbf{v}}$:

$$f \triangleq \sum_{i,j \in [5]} a_{ij} \langle \mathbf{s}_i, \mathbf{s}_j \rangle + a \langle \tilde{\mathbf{v}}_1, \tilde{\mathbf{v}}_2 \rangle + \langle \varphi, (\tilde{\mathbf{s}}, \tilde{\mathbf{v}}) \rangle + f_0 = 0, \quad (4)$$

where $a_{ij}, a \in \mathcal{R}_q$, $\varphi \in \mathcal{R}_q^{\sum_{i \in [6]} n_i + 16}$, and f_0 a linear function in h_i 's and $v_i^{(0)}$'s. After aggregation, the vectors $\mathbf{s}_3$ and $\mathbf{s}_6$ still appear only in the linear part. If one of the full constraints is not satisfied, then the final constraint f would be satisfied with probability $1/q^{d/2}$, assuming q splits into two factors.

Prove the Final Quadratic. Now the prover wants to let the verifier check Equation (4) in terms of the *masked openings* of $(\tilde{\mathbf{s}}, \tilde{\mathbf{v}})$ and $(\mathbf{r}_s, \mathbf{r}_v)$, namely $\mathbf{z}_1 = (\mathbf{z}_1^s, \mathbf{z}_1^v) := \mathbf{y}_1 + c \cdot (\tilde{\mathbf{s}}, \tilde{\mathbf{v}}) \in \mathcal{R}_q^{\sum_{i \in [6]} n_i + 16}$ and $\mathbf{z}_2 = (\mathbf{z}_2^s, \mathbf{z}_2^v) := \mathbf{y}_2 + c \cdot (\mathbf{r}_s, \mathbf{r}_v) \in \mathcal{R}_q^{n_s + n_v}$, for some challenge $c \in \mathcal{C}$ and masks $\mathbf{y}_1 \in \mathcal{R}^{\sum_{i \in [6]} n_i + 16}$ and $\mathbf{y}_2 \in \mathcal{R}^{n_s + n_v}$. Applying rejection sampling to separate masked openings of $\tilde{\mathbf{s}}$ and $\tilde{\mathbf{v}}$ would be too costly, which is why we treat them together. Let us split the $\mathbf{z}_i$'s and $\mathbf{y}_i$'s:

$$\begin{aligned} \mathbf{z}_1 &:= (\mathbf{z}_{1,1}^s, \dots, \mathbf{z}_{1,6}^s, \mathbf{z}_{1,1}^v, \mathbf{z}_{1,2}^v), \mathbf{z}_{1,i}^s \in \mathcal{R}_q^{n_i}, \mathbf{z}_{1,i}^v \in \mathcal{R}_q^8, \\ \mathbf{z}_2 &:= (\mathbf{z}_2^s, \mathbf{z}_2^v), \mathbf{z}_2^s \in \mathcal{R}_q^{n_s}, \mathbf{z}_2^v \in \mathcal{R}_q^{n_v}, \\ \mathbf{y}_1 &:= (\mathbf{y}_{1,1}^s, \dots, \mathbf{y}_{1,6}^s, \mathbf{y}_{1,1}^v, \mathbf{y}_{1,2}^v), \mathbf{y}_{1,i}^s \in \mathcal{R}_q^{n_i}, \mathbf{y}_{1,i}^v \in \mathcal{R}_q^8, \\ \mathbf{y}_2 &:= (\mathbf{y}_2^s, \mathbf{y}_2^v), \mathbf{y}_2^s \in \mathcal{R}_q^{n_s}, \mathbf{y}_2^v \in \mathcal{R}_q^{n_v}. \end{aligned}$$

For proving Equation (4), he provides the verifier some *garbage terms* $u', v' \in \mathcal{R}_q$, to make the following hold

$$\begin{aligned} &\sum_{i,j \in [5]} a_{ij} \langle \mathbf{z}_{1,i}^s, \mathbf{z}_{1,j}^s \rangle + a \langle \mathbf{z}_{1,1}^v, \mathbf{z}_{1,2}^v \rangle + c \cdot \langle \varphi, \mathbf{z}_1 \rangle + c^2 \cdot f_0 \\ &= c^2 \left(\sum_{i,j \in [5]} a_{ij} \langle \mathbf{s}_i, \mathbf{s}_j \rangle + a \langle \tilde{\mathbf{v}}_1, \tilde{\mathbf{v}}_2 \rangle + \langle \varphi, (\tilde{\mathbf{s}}, \tilde{\mathbf{v}}) \rangle + f_0 \right) + cu' + v'. \end{aligned} \quad (5)$$

To prove that the coefficient of c^2 in Equation (5) vanishes, we send a commitment u to u' and a randomized version v of v' as

$$u := u' + \langle \varphi_{\text{rand}}, \mathbf{r}_s \rangle, \quad v := v' + \langle (\varphi_{\text{rand}}, 0), \mathbf{y}_2 \rangle. \quad (6)$$

If the verifier is given u, v and the masked openings $\mathbf{z}_i$, he can check

$$\begin{aligned} & \sum_{i,j \in [5]} a_{ij} \langle \mathbf{z}_{1,i}^s, \mathbf{z}_{1,j}^s \rangle + a \langle \mathbf{z}_{1,1}^v, \mathbf{z}_{1,2}^v \rangle + c \cdot \langle \varphi, \mathbf{z}_1 \rangle + c^2 \cdot f_0 \\ &= cu + v - \langle (\varphi_{rand}, 0), \mathbf{z}_2 \rangle, \end{aligned} \quad (7)$$

which is exactly Equation (5) when the coefficient of c^2 is 0.

Now we explain the remaining steps of the protocol. The prover samples the masking vectors $\mathbf{y}_1 := (\mathbf{y}_1^s, \mathbf{y}_1^v) \leftarrow D_{\sigma_1}^{\sum_{i \in [6]} n_i + 16}$ and $\mathbf{y}_2 := (\mathbf{y}_2^s, \mathbf{y}_2^v) \leftarrow D_{\sigma_2}^{n_s + n_v}$ and computes their Ajtai commitments as

$$\mathbf{w}_s := \mathbf{A}_1 \mathbf{y}_1^s + \mathbf{A}'_1 \mathbf{y}_2^s \in \mathcal{R}_q^{k_s}, \quad \mathbf{w}_v := \mathbf{A}_2 \mathbf{y}_1^v + \mathbf{A}'_2 \mathbf{y}_2^v \in \mathcal{R}_q^{k_v}. \quad (8)$$

Then he computes u, v as in Eqs. (5) and (6). At this point, the prover could send $\mathbf{w}_s, \mathbf{w}_v, u$, and v .

Next, after receiving the challenge $c \leftarrow \mathcal{C}$ from the verifier, the prover computes $\mathbf{z}_1 := \mathbf{y}_1 + c \cdot (\tilde{\mathbf{s}}, \tilde{\mathbf{v}})$ and $\mathbf{z}_2 := \mathbf{y}_2 + c \cdot (\mathbf{r}_s, \mathbf{r}_v)$ and applies the rejection sampling from Lemma 3 (1), for each $\mathbf{z}_i, i \in [2]$. If both algorithms do not abort, the prover outputs $\mathbf{z}_1, \mathbf{z}_2$. Note that randomness $(\mathbf{r}_s, \mathbf{r}_v)$ is used only once, so a bit of leakage is allowed as when applying Lemma 3 (2). The standard deviations σ_1 and σ_2 are set in Section D.2. By rejection sampling, each $\mathbf{z}_i$ (and its splits) follows a distribution close to D_{σ_i} , so that the verifier can check if it is small enough, i.e. $\|\mathbf{z}_i\| \leq \sigma_i \sqrt{2 \dim(\mathbf{z}_i)}$, with high probability. (see Lemma 2).

Output and Checks. The last message of the prover is, in fact, made by the masked openings $\mathbf{z}_i$ and the openings of the outer commitments, which are all binary polynomials collected in the vector $\mathbf{x}_{\text{bin}} \in \mathcal{R}^{n_{\text{bin}}}$. In addition to the correctness of the commitments and the binary check on $\mathbf{x}_{\text{bin}}$, the verifier has to check the quadratic constraint and separate norm bounds on the splits of $\mathbf{z}_1$, as these appear in the dot-products of Eq. (7), along with the norm bound on $\mathbf{z}_2$:

$$\mathbf{A}_1 \mathbf{z}_1^s + \mathbf{A}'_1 \mathbf{z}_2^s = \mathbf{w}_s + \mathbf{c} \mathbf{t}_s \quad (9)$$

$$\mathbf{A}_2 \mathbf{z}_1^v + \mathbf{A}'_2 \mathbf{z}_2^v = \mathbf{w}_v + \mathbf{c} \mathbf{t}_v \quad (10)$$

$$\mathbf{c} \mathbf{t}(h_i) = 0, \quad i \in [\lambda] \quad (11)$$

$$\begin{aligned} & \sum_{i,j \in [5]} a_{ij} \langle \mathbf{z}_{1,i}^s, \mathbf{z}_{1,j}^s \rangle + a \langle \mathbf{z}_{1,1}^v, \mathbf{z}_{1,2}^v \rangle + c \cdot \langle \varphi, \mathbf{z}_1 \rangle + c^2 \cdot f_0 \\ &= cu + v - \langle (\varphi_{rand}, 0), \mathbf{z}_2 \rangle \end{aligned} \quad (12)$$

$$\|\mathbf{z}_{1,i}^s\| \leq \sigma_1 \sqrt{2 \cdot 256 n_i}, \quad i \in [6] \quad (13)$$

$$\|\mathbf{z}_{1,1}^v\|, \|\mathbf{z}_{1,2}^v\| \leq \sigma_1 \sqrt{2 \cdot 256 \cdot 8} \quad (14)$$

$$\|\mathbf{z}_2\| \leq \sigma_2 \sqrt{2 \cdot 256 \cdot (n_s + n_v)} \quad (15)$$

$$\text{bin-check}(\mathbf{x}_{\text{bin}}) \quad (16)$$

Note that the second masked opening $\mathbf{z}_2$ appears only in the linear part of Eq. (7), so there is no need to split. Moreover, all these checks are compatible with the constraint system of LaBRADOS.

Security. Here we analyze the security of LNP-Lite, at a high level, and refer the reader to Section B.2 for more details.

Completeness. Assume that the prover and the verifier behave honestly. Eqs. (9) and (10) hold thanks to Eqs. (2), (3) and (8). Equation (12) holds thanks to Equation (5). Then the verifier is convinced of the well-formedness of h_i 's, $i \in [\lambda]$, whose check Equation (11) holds, because of the masks g_i . The norm bounds hold with overwhelming probability, according to Lemma 2, as, thanks to rejection sampling (Lemma 3), $\mathbf{z}_1$ and $\mathbf{z}_2$ follow (closely) Gaussian distributions.

Zero-Knowledge. It is enough to check that the protocol with no outer commitment has zero-knowledge, thanks to Lemma 5. For this, we follow the same lines as [36, Thm 4.2], except that here we deal with two different Extended M-LWE problems, whose secrets correspond to the random vectors $\mathbf{r}_s$ and $\mathbf{r}_v$ used in hiding the (inner) commitments.

Soundness. Again, we prove knowledge soundness only for the protocol without outer commitments, thanks to Lemma 4. For simplicity, let us consider the norm bounds as $\|\mathbf{z}_1^s\| \leq B_s$, $\|\mathbf{z}_1^v\| \leq B_v$, $\|\mathbf{z}_2^s\|, \|\mathbf{z}_2^v\| \leq \|\mathbf{z}_2\| \leq B_r$. Here we state knowledge soundness. At a high level, we derive the proof by viewing the first step of the projection as a reduction of knowledge from the input relation to a new relation, which could be proven by the original LNP [36, Prop 5.1.].

Theorem 2. *Let $\mathcal{C} \subset \mathcal{R}_q$ be the challenge space from Section 2.1, of polynomials with operator norm bounded by T . Suppose M-SIS is hard for ℓ_2 norm bound $8T\sqrt{B_s^2 + B_r^2}$ and rank k_s , and for ℓ_2 norm bound $8T\sqrt{B_v^2 + B_r^2}$ and rank k_v . Moreover, assume $\max\{2\sigma\sqrt{512/26} \cdot 2048 \cdot 41, \{2^{k_i+1}/0.74 \cdot 91\}_{i \leq 4}, 2 \cdot d \cdot n_4 \cdot (\frac{2 \cdot 9.75}{0.74})^2, 2 \cdot (2\sigma\sqrt{512/26})^2\} \leq q$. Then LNP-Lite is a knowledge-sound proof with knowledge soundness $\kappa := 5/2^{128} + (1/q)^\lambda + 1/q^{d/2} + 2/|\mathcal{C}|$ and norm slack $c_{\text{LNP-Lite}} := \frac{2 \cdot 9.75}{0.74} \simeq 26.35$, i.e., the extractor is only guaranteed to extract witness vectors $\mathbf{s}_i$ of norm bounds $\|\mathbf{s}_i\| \leq 26.35\beta_i$.*

3.4 Proving the LNP-Lite Output with LaBRADOS

Now we take the LNP-Lite output, $(\mathbf{z}_1, \mathbf{z}_2, \mathbf{x}_{\text{bin}})$ and the checks it satisfies, and feed them into LaBRADOS. Note that, thanks to the zero-knowledge protocol, hiding the outer commitments of LaBRADOS is no longer needed.

Decompose. As [11], we decompose each $\mathbf{z}_i$ into two parts, with respect to some integer basis b_i , namely $\mathbf{z}_i = \mathbf{z}_i^{(0)} + b_i \mathbf{z}_i^{(1)}$, following a rationale similar to that in [11]. Notice that $\sigma_{-1}(\mathbf{x}_{\text{bin}})$ has to be part of the witness of LaBRADOS, as **bin-check** considers both $\mathbf{x}_{\text{bin}}$ and $\sigma_{-1}(\mathbf{x}_{\text{bin}})$ and further, the challenge space used in LaBRADOS is not invariant under σ_{-1} .

Minimize the Number of Witness Parts. We note that the input vectors of LNP $\mathbf{s}_3$ and $\mathbf{s}_6$ are only involved in the linear parts of the aggregated quadratic f . Thus, we could think of concatenating (the parts of) their masking openings $\mathbf{z}_{1,3}^s$ and $\mathbf{z}_{1,6}^s$ together, along with the (the parts of) masked opening $\mathbf{z}_2$. Moreover, Equation (12) considers the dot-products between different parts of the masked openings, namely $\mathbf{z}_{1,i}^s$, for $i \in \{1, 2, 4, 5\}$, $\mathbf{z}_{1,1}^v, \mathbf{z}_{1,2}^v$, so these parts should be considered as separate witness vectors for LaBRADOS. In addition to this, we also consider further improvements. Thus, we feed LaBRADOS with the witness made of the parts of each vector

$$\mathbf{z}_{1,1}^s, \mathbf{z}_{1,2}^s, \mathbf{z}_{1,4}^s, \mathbf{z}_{1,5}^s, \mathbf{z}_{1,1}^v, \mathbf{z}_{1,2}^v, (\mathbf{z}_{1,3}^s, \mathbf{z}_{1,6}^s, \mathbf{z}_2),$$

along with $\mathbf{x}_{\text{bin}}, \sigma_{-1}(\mathbf{x}_{\text{bin}})$. The statement that this witness satisfies is created by adapting the verifier checks (Equations (9) to (12)), when replacing the masked openings with their decompositions, and the norm checks updated as before, and on the binary vector $\|\mathbf{x}_{\text{bin}}\| \leq \sqrt{d \cdot n_{\text{bin}}}$. We use the fact that $\langle \mathbf{z}^{(0)} + b\mathbf{z}^{(1)}, \mathbf{z}^{(0)} + b\mathbf{z}^{(1)} \rangle = \langle \mathbf{z}^{(0)}, \mathbf{z}^{(0)} \rangle + b \cdot (\langle \mathbf{z}^{(0)}, \mathbf{z}^{(1)} \rangle + \langle \mathbf{z}^{(1)}, \mathbf{z}^{(0)} \rangle) + b^2 \cdot \langle \mathbf{z}^{(1)}, \mathbf{z}^{(1)} \rangle$.

3.5 The Security of the Pack

We analyze the security of the entire construction. Completeness follows as each iteration of it, LaBRADOS, LNP-Lite and LaBRADOR, satisfies completeness. Soundness follows by coupling the soundness of each iteration, via [11, Lem.3.7.], and the soundness of each iteration follows by Theorem 1 for LaBRADOS, Theorem 2 for LNP-Lite and by [11, Thm.5.1] for LaBRADOR, each combined with Lemma 4 (when extending with outer commitments). Zero-knowledge is straightforward. To simulate a proof, the simulator can first simulate the prover messages from the iterations of LaBRADOS before LNP-Lite as commitments of 0, which, by hiding the outer commitments, are indistinguishable from the real ones. When getting to LNP-Lite, he can first simulate the last message of LNP-Lite and compute the outer commitments of everything from this message, except of the masked openings, which become the remaining LNP prover's messages. Then, when entering the next iterations of LaBRADOS and LaBRADOR, he can simply honestly compute the remaining transcript of the composition, feeding them with the simulated LNP-Lite output. We discuss the security of the non-interactive version in Section C.

4 Zero-Knowledge in Practice: Case Studies

We evaluate the performance of our proof system from Section 3 by implementing relevant benchmark examples. To this end, we describe our instantiations of some known primitives, that we tailor to the constraints our proof system supports.

As a first example, detailed in Section 4.1, we describe a construction of *verifiable symmetric encryption* built upon a pseudorandom generator, obtained from iterative *learning with rounding*. We note that one can use such a scheme in practice with a public key verifiable encryption scheme which encrypts the PRG seed to construct a very efficient public key verifiable encryption scheme that encrypts long messages. Second, in Section 4.2, we employ a variant of Ajtai’s collision-resistant hash that allows for committing to large inputs (in a streaming manner), and for membership proofs (via Merkle trees). In Section 4.3, we describe a verifiable *mixing subroutine* that, combined with the previous building blocks, leads to a potential candidate of a *full-fledged cryptographic hash function*.⁵ We choose to benchmark *blind signatures* as one of the useful examples that the verifiable hash enables. We also describe, for theoretical interest, how one can use such a hash to construct *Incrementally Verifiable Computation* (IVC), which is not yet practically achieved by *any* lattice-based constructions.

For all the use cases (except IVC), we provide implementations on top of the LaZer library [40] and report the prover and verifier timings, both when zero-knowledge is enabled (labelled as “zk”) and when it is not (labelled as “not zk”). The secure version is the zero-knowledge-enabled one. The reason why we also include the timings without zero-knowledge is to assess the overhead of adding zero-knowledge to the proof. All the timings come from running the benchmarks on a single core of an Intel Tiger Lake-H CPU.

We remark that the proof size remains quite constant across use cases and input size, since the proof size is dominated by the last round of LaBRADOR and therefore we do not report them explicitly. The proof size for non-zero-knowledge examples is around 100KB, whereas the zero-knowledge enabled ones result in around 110KB. The 10KB difference comes from the commitments sent in LNP-Lite and in the extra rounds of LaBRADOS that come after LNP-Lite. This relatively small constant overhead in the proof size is achieved thanks to the strategy of running LNP-Lite only once the input is small enough and compressing its messages.

Throughout this section, we work with the polynomial ring $\mathcal{R} = \mathbb{Z}[X]/(X^d + 1)$ with $d = 512$. All the operations in this ring could be translated as operations over the proof system ring (with degree $d = 256$), according to [37, Sec.2.8].

4.1 Pseudorandom Generator

The goal of the pseudorandom generator (PRG) is to produce many pseudorandom polynomials (output) from a few random ones (seed). The idea is to apply an expansion function and use part of its output as a new seed and part to compose the PRG output. In this construction, the pseudorandomness of the computed values is based on the hardness of the M-LWR problem.

In detail, the expansion function takes as input two binary polynomials $x_0, x_1 \in \mathcal{R}_2$, computes

$$\lceil \mathbf{A}(x_0, x_1)^T \rceil_{256 \rightarrow 64}$$

for $\mathbf{A} \leftarrow \mathcal{R}_{256}^{1 \times 2}$, and binary decomposes the result. As the rounding drops the two least significant bits, we write with abuse of notation $\lceil \mathbf{A}(x_0, x_1)^T \rceil_{256 \rightarrow 64} = \mathbf{G}_2(y^{(2)}, \dots, y^{(7)})$ with $\mathbf{G}_2 = (1, 2, \dots, 2^5)$. Let $s_0^{(6)}, s_0^{(7)} \in \mathcal{R}_2$ be the seed. The pseudorandom generator works as follows:

$$\begin{aligned} \lceil \mathbf{A}(s_0^{(6)}, s_0^{(7)})^T \rceil_{256 \rightarrow 64} &= \mathbf{G}_2(s_1^{(2)}, \dots, s_1^{(7)})^T \\ \lceil \mathbf{A}(s_1^{(6)}, s_1^{(7)})^T \rceil_{256 \rightarrow 64} &= \mathbf{G}_2(s_2^{(2)}, \dots, s_2^{(7)})^T \\ &\vdots \end{aligned} \tag{17}$$

⁵ We have not performed any cryptanalysis of this function, so it should be viewed as a possible line of research for further study orthogonal to the main results of the paper.

The output is $\mathbf{s} := (s_1^{(2)}, s_1^{(3)}, s_1^{(4)}, s_1^{(5)}, s_2^{(2)}, \dots)^T$.

Proof-friendly. Note that the expansion equation can be written as $\mathbf{A}(s_i^{(6)}, s_i^{(7)})^T = \mathbf{G}'_2(s_{i+1}^{(0)}, \dots, s_{i+1}^{(7)})^T \bmod 256$ with $\mathbf{G}'_2 = (1, -2, 2^2, \dots, 2^7)$, where -2 is required to write the product coefficients in $[-2, 253]$ and correctly drop the two least significant bits. To prove correct execution of the PRG, one needs to prove knowledge of the input $s_0^{(6)}, s_0^{(7)}$ and the intermediate witnesses $s_i^{(0)}, \dots, s_i^{(7)}$ that are binary and satisfy $\mathbf{A}(s_i^{(6)}, s_i^{(7)})^T = \mathbf{G}'_2(s_{i+1}^{(0)}, \dots, s_{i+1}^{(7)})^T \bmod 256$. This is natively proven by the proof system in Section 3, since that the statement can be written over a ring $\mathcal{R}_q$ with sufficiently large modulo q , as $\mathbf{A}(s_i^{(6)}, s_i^{(7)})^T - 256l_i = \mathbf{G}'_2(s_{i+1}^{(0)}, \dots, s_{i+1}^{(7)})^T$ for some $l_i \in \mathcal{R}$.

Security. This function is a pseudorandom generator under the hardness assumption of some Module-LWR problem. Informally, an adversary distinguishing a PRG output with advantage ϵ distinguishes with advantage ϵ/k a pair of successive hybrids, hence an instance of the Module-LWR problem with parameters $q = 256$, $p = 64$, $m' = 1$, $n' = 2$, $\chi = \mathcal{U}(\mathcal{R}_2)$, whose concrete hardness we analyze with [4]. The full proof is deferred to Section E.2.

Output transformations. The output obtained at this point is a binary vector over the ring $\mathcal{R}$ of degree $d = 512$. However, one may target uniformly random vectors over the quotient ring $\mathcal{R}_q$ for some $q > 2$, or of a different degree d' .

To obtain a uniformly random vector $\mathbf{t} \in \mathcal{R}_q^l$, one needs a binary output of the PRG $\mathbf{s} \in \mathcal{R}_2^{l_\ell}$ with $\ell := \lceil \log_2 q \rceil + 1$. A polynomial t_i is obtained by multiplying a block of the binary vector, $\tilde{\mathbf{s}}_i \in \mathcal{R}_2^\ell$, by a matrix $\mathbf{B} = (\mathbf{B}', \mathbf{I}_1)$ where $\mathbf{B}' \leftarrow \mathcal{R}_q^{1 \times \lceil \log_2 q \rceil}$:

$$\mathbf{t}^T = (t_0, t_1, \dots) = \mathbf{B} \cdot (\tilde{\mathbf{s}}_0, \tilde{\mathbf{s}}_1, \dots) \bmod q.$$

The security follows from the hardness of some M-LWE problem with the hybrid argument. We defer its proof to Section E.3.

To obtain an element in a ring of different degree, one can use the isomorphism described in [37, Sec.2.8].

Verifiable Symmetric Encryption Pseudorandom generators are often used in stream ciphers, where, given a binary plaintext m and a PRG-expanded key, the ciphertext c is obtained as $c = m \oplus \text{PRG}(k)$. Noticing that this equation can be equivalently written as $c = m + \text{PRG}(k) \bmod 2$, one can achieve verifiable symmetric encryption by proving knowledge of binary plaintext m , key k , and lifting l satisfying $c - 2l = m + \text{PRG}(k)$ over the proof system ring.

In Table 1, we present the results of benchmarking the expansion component of that construction. In particular, we expand a small seed of fixed-length (1024 bits) into outputs of increasing length, and use our proof system to prove the correctness of that expansion.

Output Dimension	Prover		Verifier	
	zk	not zk	zk	not zk
2^6	1.04	0.12	0.51	0.1
2^8	1.01	0.22	0.51	0.17
2^{10}	1.51	0.6	0.7	0.35
2^{12}	2.87	1.87	1.27	0.91
2^{14}	7.86	6.78	3.09	2.69
2^{16}	27.87	26.57	9.69	9.3

Table 1. Timing results in seconds for proving the **expansion** of a seed into vectors of binary polynomials. The dimension refers to the number of output polynomials of degree 512.

4.2 Collision Resistant Hash Function

The hash function described in the following is based on the Merkle-Damgård construction to build collision-resistant hash functions [23,41]. Informally, the arbitrary-length input is divided in blocks, and a compression function is applied iteratively to them to obtain a fixed-length output. If the compression function is collision resistant, then the hash function is also collision resistant. In this construction, collision resistance is based on the hardness of the M-SIS problem.

In details, let $\mathbf{x}$ be an arbitrary-length binary input, written as a polynomial vector over the ring $\mathcal{R}$ ⁶. Firstly, a padding is applied to make the input divisible into blocks. It consists of the minimum number of zero polynomials, followed by one polynomial encoding the dimension of the input in binary:

$$\text{padded}(\mathbf{x}) = (\mathbf{x}, \mathbf{0}, \text{poly}(\dim(\mathbf{x}))).$$

The compression function is composed by matrix multiplication over $\mathcal{R}_{256}$, followed by binary decomposition. Let $\mathbf{A} \leftarrow \mathcal{R}_{256}^{1 \times 16}$, we define the compression function $f : \mathcal{R}_2^{16} \rightarrow \mathcal{R}_2^8$ as

$$\mathbf{x} \mapsto \mathbf{y} := \text{BinDecomp}(\mathbf{A}\mathbf{x} \bmod 256).$$

With abuse of notation, we will denote this operation as $\mathbf{A}\mathbf{x} = \mathbf{G}_2\mathbf{y} \bmod 256$ with $\mathbf{G}_2 := (1, 2, \dots, 2^7)$. Finally, let $\mathbf{x}_0, \mathbf{x}_1, \dots \in \mathcal{R}_2^8$ be the blocks of 8 polynomials composing the padded input, and let $\text{IV} \in \mathcal{R}_2^8$ be a fixed initial vector. We compute as follows:

$$\begin{aligned} \mathbf{A}(\text{IV}, \mathbf{x}_0) &= \mathbf{G}_2\mathbf{y}_0 \bmod 256 \\ \mathbf{A}(\mathbf{y}_0, \mathbf{x}_1) &= \mathbf{G}_2\mathbf{y}_1 \bmod 256 \\ &\vdots \end{aligned} \tag{18}$$

The output is the last value of $\mathbf{y}_i$.

Proof-friendly. To prove knowledge of a preimage of the hash function, one needs to prove knowledge of the input blocks $\mathbf{x}_i$ and the intermediate witnesses $\mathbf{y}_i$ that are binary and satisfy $\mathbf{A}(\mathbf{y}_{i-1}, \mathbf{x}_i) = \mathbf{G}_2\mathbf{y}_i \bmod 256$. The hash preimage is natively proven by the proof system in Section 3, noticing that the last statement can be written over a ring $\mathcal{R}_q$ with sufficiently large modulo q , as $\mathbf{A}(\mathbf{y}_{i-1}, \mathbf{x}_i) - 256l_i = \mathbf{G}_2\mathbf{y}_i$ for some $l_i \in \mathcal{R}$.

Security. The hash function is collision resistant under the hardness assumption of some Module-SIS problem, whose concrete hardness is more than 256-bits [4]. At a high level, this is because from an adversary that finds collisions in the hash function, one constructs an adversary that finds collisions in the compression function, namely pairs $(\mathbf{y}_{i-1}, \mathbf{x}_i) \neq (\mathbf{y}'_{i-1}, \mathbf{x}'_i)$ such that $\mathbf{A}(\mathbf{y}_{i-1}, \mathbf{x}_i) = \mathbf{A}(\mathbf{y}'_{i-1}, \mathbf{x}'_i) \bmod 256$, and obtains a solution to the M-SIS problem. The full proof is deferred to Section E.1.

Committing to Large Inputs The hash function presented above can be used as an alternative to the Ajtai commitment for inputs of large dimension. The advantage is that the public key and the modulus are smaller and fixed. We show results in Table 2 when using our proof system to prove correctness of such commitments to inputs of increasing length.

Batched Proof of Membership Suppose that we want to prove knowledge of a private element contained in a public set S consisting of 2^N elements (e.g. the public keys of users). We can do so by proving knowledge of a path in a Merkle tree of height N computed as follows:

- The leaves are the hashes of the elements in the set.
- The parent node $\mathbf{y} \in \mathcal{R}_2^8$ is computed from the children $\mathbf{x}_l, \mathbf{x}_r \in \mathcal{R}_2^8$ using the compression function f as $\mathbf{A}(\mathbf{x}_l, \mathbf{x}_r) = \mathbf{G}_2\mathbf{y} \bmod 256$.

⁶ If the input is originally not in this ring, a padding may be required.

Input Dimension	Prover		Verifier	
	zk	not zk	zk	not zk
2^6	1.13	0.13	0.53	0.1
2^8	1.26	0.32	0.56	0.21
2^{10}	1.84	0.68	0.74	0.39
2^{12}	2.8	1.93	1.21	0.89
2^{14}	7.16	6.29	2.8	2.48
2^{16}	29.35	29.44	9.18	9.58

Table 2. Timing results in seconds for proving the **compression** of vectors of polynomials. The dimension refers to the number of input polynomials of degree 512.

To prove knowledge of a path in the tree from the root to the leaf corresponding to the private element in zero-knowledge, one proves knowledge of the nodes $\mathbf{v}_i \in \mathcal{R}_2^8$, their siblings $\mathbf{w}_i \in \mathcal{R}_2^8$, and the path indicators $x_i \in \{0, 1\}$ (encoding 0 if $\mathbf{v}_i$ is the left child, 1 otherwise) satisfying, for any level index i ,

$$(1 - x_i) \cdot \mathbf{A}(\mathbf{v}_i, \mathbf{w}_i) + x_i \cdot \mathbf{A}(\mathbf{w}_i, \mathbf{v}_i) = \mathbf{G}_2 \mathbf{v}_{i-1} \bmod 256.$$

Additionally, all witnesses must be binary, and the path indicators also be integers.

In our results in Table 3, we consider a tree of height 64 and prove knowledge of several private elements belonging to the tree.

Number of Paths	Prover		Verifier	
	zk	not zk	zk	not zk
1	1.61	0.56	0.7	0.33
2	1.68	0.82	0.82	0.46
4	2.39	1.37	1.09	0.72
8	3.37	2.23	1.48	1.06
16	5.22	4.39	2.2	1.91
32	9.37	8.59	3.97	3.68
64	17.85	17.62	6.35	6.32
128	39.2	35.73	12.63	11.61

Table 3. Timing results in seconds for proving **membership** of increasing numbers of elements in a tree of height 64.

4.3 Beyond Collision Resistance

In this section, we attempt to construct a cryptographic hash function candidate with the intention of instantiating a random oracle with this candidate primitive. This hash function relies on the *mixing* subroutine, a function that behaves as much as possible like a truly random one. We present how this instantiation gives rise to a construction of a blind signature.

Mixing The goal of the mixing is to obtain a function that behaves as much as possible like a truly random one, and can be used in practice to instantiate it (eventually combined with other components).

Let the input vector be $\mathbf{z}_0 \in \mathcal{R}_2^9$. The mixing consists of applying a fixed number of times two functions, described by public parameters $\mathbf{A} \leftarrow \mathcal{R}_{256}^{1 \times 9}$, $\mathbf{A}' \leftarrow \mathcal{R}_{257}^{1 \times 8}$, and $c_0, \dots, c_4 \leftarrow \mathcal{R}_{256}$. Let $\mathbf{G}_2 = (1, 2, \dots, 2^7)$,

$\mathbf{G}'_2 = (1, 2, \dots, 2^8)$. Then

$$\begin{aligned}
\mathbf{A}\mathbf{z}_0 + c_0 &= \mathbf{G}_2\mathbf{z}'_1 \pmod{256} \\
\mathbf{A}'\mathbf{z}'_1 &= \mathbf{G}'_2\mathbf{z}_1 \pmod{257} \\
\mathbf{A}\mathbf{z}_1 + c_1 &= \mathbf{G}_2\mathbf{z}'_2 \pmod{256} \\
&\vdots \\
\mathbf{A}'\mathbf{z}'_4 &= \mathbf{G}'_2\mathbf{z}_4 \pmod{257} \\
\mathbf{A}\mathbf{z}_4 + c_4 &= \mathbf{G}_2\mathbf{z}'_5 \pmod{256}
\end{aligned} \tag{19}$$

If the mixing achieves its scope, in practice, it can only be validated by cryptanalysis; however, in the following, we give the intuition behind its components. The matrix multiplication provides collision resistance and *confusion*, namely, makes the relation between input and output as complex as possible. The binary decomposition yields *diffusion*, spreading the influence of an input polynomial across many output ones; additionally, it introduces some non-linearity. The alternating moduli aim at further avoiding patterns that an adversary could exploit in distinguishing the output from random. Finally, the public constants c_i ensure that, at any iteration, the zero input is not assigned to the zero output.

Proof-friendly. Due to the operations over the ring $\mathcal{R}_{257}$, proving the mixing relation additionally requires ensuring that the element $\mathbf{z}_i$ is the binary representation of a polynomial modulo 257, i.e. that either the most significant bit is 0, or all the others are. This can be proven by imposing that the sums $z_i^{(0)} + z_i^{(8)}, \dots, z_i^{(7)} + z_i^{(8)}$ are binary.

Cryptographic Hash Function Candidate Many protocols involving hash functions prove security in the random oracle model (ROM), [7], namely, treating a hash function as a truly random function. One could attempt to construct such a candidate hash function by combining the previous building blocks. First, the collision resistant hash function in Section 4.2 is applied to compress the arbitrary-length input. Second, we apply a “mixing component” described in in Section 4.3; however, one could use any alternative instantiation of a random function. Finally, we can apply the pseudo-random generator in Section 4.1. It can be proved that if the mixing is simulated by a random oracle, the candidate full-fledged hash function is pseudorandom. Our “compress-mix-expand” cryptographic hash function construction cannot be proven secure in the indistinguishability framework (c.f. [20]) by just viewing the mixing part as a random oracle. Nevertheless, we believe that because the lattice-based compression and expansion components are well-understood cryptographic constructions, one could try to cryptanalyze the function as a whole. We leave this line of research to future work.

Blind signatures Blind signatures are protocols in which the user obtains signatures of their messages without revealing to the signer any information about the message. In [10], the authors propose a blind signature scheme from lattice assumptions, whose security is proven in the random oracle model. At a high level, in this protocol the user commits to the message μ computing $\mathbf{c} = \mathbf{B}\mathbf{r} + H(\mu, H(\mathbf{r})) \in \mathcal{R}_q$ with commitment matrix $\mathbf{B} \leftarrow \mathcal{R}_q^{1 \times 2}$ and randomness $\mathbf{r} \leftarrow \chi^2$, and sends to the signer the commitment $\mathbf{c}$ with a zero-knowledge proof of knowledge for its correctness. The issuer signs the commitment providing a short preimage $\mathbf{s}$, i.e. such that $\mathbf{A}\mathbf{s} = \mathbf{c}$. The user proves that they have a valid signature for μ , with a zero-knowledge proof of knowledge for $\mathbf{s}, \mathbf{r}$ such that $\mathbf{A}\mathbf{s} = \mathbf{B}\mathbf{r} + h$. While this linear proof can be efficiently done with the original LNP, proving the correctness of the commitment $\mathbf{c}$ benefits from the proof system in Section 3. In the following, we benchmark this proof, when instantiating the hash function with the hash function candidate presented in Section 4.3.

Incrementally Verifiable Computation from Folding Scheme [15, 16, 18, 19, 26, 29, 33, 43] propose quantum-secure folding schemes (or accumulation schemes) that fold witness-statement pairs for a relation into a single pair. Folding schemes are used as building blocks for incrementally verifiable computation

Message Dimension	Prover		Verifier	
	zk	not zk	zk	not zk
10	1.18	0.26	0.5	0.18

Table 4. Timing results in seconds for proving the knowledge of a commitment in the blind signature protocol. The dimension refers to the number of input polynomials of degree 512.

protocols (IVC) [45] and proof-carrying data (PCD) [22]. Both primitives follow a general blueprint: the verifier circuit (including hash functions after the Fiat–Shamir transformation) is expressed as a set of constraints for an input relation, and the folding-scheme prover recursively folds the witness-statement pairs of that verifier circuit at each computation step. A key advantage is that the proof size and verification time of IVC/PCD are independent of the number of computation steps, and the memory required by the prover is also step-independent, unlike “monolithic” SNARKs (following the terminology in, e.g., [17]). A crucial component, and the main reason current constructions remain impractical, is the hash function embedded in the verifier circuit, which must be efficiently verifiable.

5 Implementation

We have implemented the benchmarks reported in Section 4 on top of the LaZer library [40]. They can be reproduced at

<https://github.com/lazer-crypto/lazer>.

Those benchmarks depend on two core modules that we have implemented: the proof system of this work, accessible at <https://github.com/lazer-crypto/labrador>; and a C library for the hash function of Section 4.3, located in the `src/lattice-hash` folder of the `lazer-crypto/lazer` GitHub repository.

5.1 Proof System

We implement the proof system of this work on top of the C arithmetic and protocols of [13], which in turn builds on top of [11, 40, 42]. We describe here our main contributions.

Dachshundv2. In practice, the statement that a user wants to prove may not be directly compatible with the statements that LaBRADOR can prove. The reason for this is that this protocol expects a small number of witness vectors for efficiency, and there is some pre-processing required to prove exact ℓ_2 -norms and binary constraints. This was addressed by the LaZer library [40] with the **Dachshund** frontend. We re-implement it to make it compatible with LaBRADOS and extend it in several fronts, resulting in the protocol that we refer to here as **Dachshundv2**. Therefore, one can see **Dachshundv2** as the frontend protocol whose output is fed to the first iteration of LaBRADOS in Figure 1.

First, in addition to exact ℓ_2 -norms and binary, the user can indicate instead that the ℓ_2 -norm of a vector is to be proven approximately, giving both the public bound it satisfies and the public bound that the proof system should guarantee. **Dachshundv2** will then merge many witness vectors, as long as the norm guarantee resulting from projecting all of them together in LaBRADOS is small enough for all the vectors involved (also accounting for the slack from the projection). This makes the proof more efficient, since proving exact norms requires proving a quadratic over the integers, and thus a larger modulus to ensure there is no wrap-around in the quadratic.

Second, we support arbitrary quadratics between witness vectors. We do so by committing to the witness and obtaining polynomial challenges that can aggregate the quadratic equations, in a similar fashion to how **Dachshund** deals with exact ℓ_2 -norms (see [40, Section 4]). We then extend the witness with two witness vectors that contain the concatenation of the left and right parts of all the quadratic terms, multiplying one of the sides by the polynomial challenges. This means that, after proving the consistency of these new parts with respect to the original witness, the original quadratics can be proven by simply proving the dot-product between these two new witness vectors. As a consequence, the original witness vectors appearing

in quadratics can safely be merged together, as long as their norm requirements allow it. Additionally, we support quadratics taking place over rings with a higher degree than that of the proof system by expanding one of the sides of the dot product into the rotation matrix and aggregating its rows with challenges before adding it to the new witness vectors (see [37, Section 2.8.] for expressing multiplication over higher-degree rings in terms of a rotation matrix).

LNP-Lite. Both the original LNP construction and its implementation are not suitable for use as a round in our composed protocol. The conceptual changes and new techniques are described in Sections 3.2 and 3.3, with the two-step projection being the core improvement. Those improvements allow **LNP-Lite** to use the same small modulus as **LaBRADOR**, allowing us to base our implementation on **LaBRADOR**’s AVX-512 optimized single-precision coefficient arithmetic. To enable protocol composition, the verifier was expressed in dot-product constraints using **LaBRADOR**’s constraint library. Both **LNP-Lite**’s input and output are **LaBRADOR**-compatible constraint systems (plus prover messages). The generation of **LNP-Lite** parameters plays a crucial role in determining the round structure of the composed protocol: if it is not possible to find the secure **LNP-Lite** parameters, more preceding **LaBRADOS** rounds will be added.

Pack. We implement a **Pack** logic that orchestrates the composition of the different protocols. Its parameter generation decides how many iterations to perform of each protocol and in which order, according to the flow of Figure 1. By default, iterations of **LaBRADOR** and **LaBRADOS** are added as long as the improvement in witness size is more than 10%. Internally, the parameter generations of those two protocols look for the split of the witness that leads to the smallest output witness. At the end of the protocol, the last iteration of **LaBRADOR** is the special last round described in [11] where no middle (nor outer) commitments are computed. When **LaBRADOS** is run before **LNP-Lite**, then the former is indicated to use randomness in the outer commitments, and further iterations are run until **LNP-Lite** successfully finds secure parameters. Additional complexity comes from the fact that the parametrization of **LaBRADOS** changes depending on whether the protocol that comes after is **LaBRADOS**, **LNP-Lite** or **LaBRADOR**, since each of them proves the ℓ_2 -norms with different slacks. The inclusion of the **LNP-Lite** round in between **LaBRADOS** rounds is optional, and determined by a flag of whether the proof requires zero-knowledge.

Python Wrapper. **LaZer** provides a convenient way of defining the statement and to populate the witness that is then fed either to **LaBRADOR** or to **LNP**. We extend **LaZer** so that the proof system used underneath is the composition of protocols described in Section 3, with an optional zero-knowledge flag that determines whether a round of **LNP-Lite** is performed or not. We additionally extend the Python API to support the definition of quadratics and approximate ℓ_2 -norms, powered by our new **Dachshundv2**.

5.2 Lattice Hash

The proof-friendly hash of Section 4.3 is realised as a stand-alone C library exposing compression, mixing, and expansion as separate entry points, with padding left to the caller. The library is independent of the rest of the toolkit and can be used externally; a companion Python module ships inside **LaZer**, integrating the construction with the proof-system frontend.

The hash is defined over $\mathcal{R} = \mathbb{Z}[X]/(X^d + 1)$ with $d = 512$. Multiplications are evaluated in $\mathcal{R}_q$ for an auxiliary prime q chosen large enough that no reduction modulo q ever occurs on a reused result, and small enough to admit a length- d negacyclic NTT and Intel HEXL’s AVX-512 modular kernels [12]. The choice of q is a coding artefact and does not affect any output. Non-NTT stages, namely splitting modulo 256 or 257 and bit decomposition, are vectorised with AVX-512.

Compression (described in Section 4.2) applies a fixed public matrix $\mathbf{A} \in \mathcal{R}^{1 \times 16}$ to the concatenation of two binary inputs $\mathbf{x} \in \{0, 1\}^{16d}$, with $\mathbf{A}$ precomputed in NTT form. Each output coefficient is split into a remainder $\mathbf{r}$ and quotient $\mathbf{q}$; the surrounding proof system has no native primitive for reducing modulo 256, so the rounding is tracked by committing to both, allowing $\mathbf{A}\mathbf{x} = 256 \cdot \mathbf{q} + \mathbf{r}$ to be enforced as a constraint in the ambient ring. Mixing (described in Section 4.3) alternates two variants of the same primitive differing only in rounding modulus: one reduces modulo 256, producing eight binary output layers (with a public shift selected by round index), the other modulo 257, producing nine. The latter is implemented with a precomputed Shoup multiplier for $257^{-1} \bmod q$, avoiding per-coefficient division. Expansion (done via PRG from Section 4.1) reuses the same NTT and splitting machinery: each iteration computes $\mathbf{a}_1\mathbf{s}_1 + \mathbf{a}_2\mathbf{s}_2$, rounds

modulo 256, and bit-decomposes the result, feeding two bit-planes into the next iteration and emitting the rest as digest material.

All public matrices and shifts are derived deterministically from a single fixed published seed via SHAKE128, isolating all randomness in one string and enabling independent audit. All primitives are data-oblivious: HEXL’s NTT and elementwise kernels, the custom AVX-512 helpers for splitting and bit decomposition, and the Shoup-based reduction modulo 257 all run in time independent of operands, with no input-dependent branches or memory accesses.

A SageMath transcript checker recomputes each stage symbolically in $\mathbb{Z}[X]/(X^d + 1)$ as an independent correctness oracle. The C library, checker, and parameter generator are self-contained, and the construction can be used as a Fiat–Shamir or Merkle-tree hash outside of LaZer.

Acknowledgements

This work was supported by the ERC Consolidator Grant PLAZA (101002845), the ERC Starting Grant GLAZE (10122273), Research Council of Finland (358951) and Technology Industries of Finland Centennial Foundation’s Future Makers 2025 grant (project SPICED).

References

1. Aardal, M.A., Aranha, D.F., Boudgoust, K., Kolby, S., Takahashi, A.: Aggregating falcon signatures with LaBRADOR. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part I. LNCS, vol. 14920, pp. 71–106. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68376-3_3
2. Albrecht, M.R., Curtis, B.R., Deo, A., Davidson, A., Player, R., Postlethwaite, E.W., Virdia, F., Wunderer, T.: Estimate all the LWE, NTRU schemes! In: Catalano, D., De Prisco, R. (eds.) SCN 18. LNCS, vol. 11035, pp. 351–367. Springer, Cham (Sep 2018). https://doi.org/10.1007/978-3-319-98113-0_19
3. Albrecht, M.R., Davidson, A., Deo, A., Gardham, D.: Crypto dark matter on the torus - oblivious PRFs from shallow PRFs and TFHE. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part VI. LNCS, vol. 14656, pp. 447–476. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58751-1_16
4. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *J. Math. Cryptol.* **9**(3), 169–203 (2015), <http://www.degruyter.com/view/j/jmc.2015.9.issue-3/jmc-2015-0016/jmc-2015-0016.xml>
5. Attema, T., Lyubashevsky, V., Seiler, G.: Practical product proofs for lattice commitments. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part II. LNCS, vol. 12171, pp. 470–499. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56880-1_17
6. Banaszczyk, W.: New bounds in some transference theorems in the geometry of numbers. In: *Mathematische Annalen*. vol. 296, p. 625–635 (1993)
7. Bellare, M., Rogaway, P.: Random oracles are practical: A paradigm for designing efficient protocols. In: Denning, D.E., Pyle, R., Ganesan, R., Sandhu, R.S., Ashby, V. (eds.) ACM CCS 93. pp. 62–73. ACM Press (Nov 1993). <https://doi.org/10.1145/168588.168596>
8. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive*, Report 2018/046 (2018), <https://eprint.iacr.org/2018/046>
9. Ben-Sasson, E., Chiesa, A., Riabzev, M., Spooner, N., Virza, M., Ward, N.P.: Aurora: Transparent succinct arguments for R1CS. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part I. LNCS, vol. 11476, pp. 103–128. Springer, Cham (May 2019). https://doi.org/10.1007/978-3-030-17653-2_4
10. Beullens, W., Lyubashevsky, V., Nguyen, N.K., Seiler, G.: Lattice-based blind signatures: Short, efficient, and round-optimal. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) ACM CCS 2023. pp. 16–29. ACM Press (Nov 2023). <https://doi.org/10.1145/3576915.3616613>
11. Beullens, W., Seiler, G.: LaBRADOR: Compact proofs for R1CS from module-SIS. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part V. LNCS, vol. 14085, pp. 518–548. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38554-4_17
12. Boemer, F., Kim, S., Seifu, G., de Souza, F.D.M., Gopal, V.: Intel HEXL: Accelerating homomorphic encryption with intel AVX512-IFMA52. *Cryptology ePrint Archive*, Report 2021/420 (2021), <https://eprint.iacr.org/2021/420>
13. Bolboceanu, M., Bootle, J., Lyubashevsky, V., Merino-Gallardo, A., Seiler, G.: Orthus: Practical sublinear batch-verification of lattice relations from standard assumptions. *Cryptology ePrint Archive*, Report 2026/398 (2026), <https://eprint.iacr.org/2026/398>

14. Bolboceanu, M., Costache, A., Hales, E., Player, R., Rosca, M., Titiiu, R.: Designs for practical SHE schemes based on ring-LWR. *CiC* **2**(1), 21 (2025). <https://doi.org/10.62056/av7tudy6b>
15. Boneh, D., Chen, B.: LatticeFold: A lattice-based folding scheme and its applications to succinct proof systems. In: ASIACRYPT 2025, Part III. pp. 330–362. LNCS, Springer, Singapore (Dec 2025). https://doi.org/10.1007/978-981-95-5099-9_11
16. Boneh, D., Chen, B.: LatticeFold+: Faster, simpler, shorter lattice-based folding for succinct proof systems. In: CRYPTO 2025, Part VII. pp. 327–361. LNCS, Springer, Cham (Aug 2025). https://doi.org/10.1007/978-3-032-01907-3_11
17. Bünz, B., Chiesa, A., Fenzi, G., Wang, W.: Linear-time accumulation schemes. Cryptology ePrint Archive, Report 2025/753 (2025), <https://eprint.iacr.org/2025/753>
18. Bünz, B., Mishra, P., Nguyen, W., Wang, W.: Accumulation without homomorphism. In: ITCS 2025. pp. 23:1–23:25. LIPIcs (Jan 2025). <https://doi.org/10.4230/LIPIcs.ITCS.2025.23>
19. Bünz, B., Mishra, P., Nguyen, W., Wang, W.: Arc: Accumulation for reed-solomon codes. In: CRYPTO 2025, Part VII. pp. 128–160. LNCS, Springer, Cham (Aug 2025). https://doi.org/10.1007/978-3-032-01907-3_5
20. Canteaut, A., Fuhr, T., Naya-Plasencia, M., Paillier, P., Reinhard, J.R., Videau, M.: A unified indistinguishability proof for permutation- or block cipher-based hash functions. Cryptology ePrint Archive, Paper 2012/363 (2012), <https://eprint.iacr.org/2012/363>
21. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, P., Ward, N.P.: Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part I. LNCS, vol. 12105, pp. 738–768. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45721-1_26
22. Chiesa, A., Tromer, E.: Proof-carrying data and hearsay arguments from signature cards. In: Yao, A.C.C. (ed.) ICS 2010. pp. 310–331. Tsinghua University Press (Jan 2010)
23. Damgård, I.: A design principle for hash functions. In: Brassard, G. (ed.) CRYPTO’89. LNCS, vol. 435, pp. 416–427. Springer, New York (Aug 1990). https://doi.org/10.1007/0-387-34805-0_39
24. Ducas, L., Durmus, A., Lepoint, T., Lyubashevsky, V.: Lattice signatures and bimodal Gaussians. In: Canetti, R., Garay, J.A. (eds.) CRYPTO 2013, Part I. LNCS, vol. 8042, pp. 40–56. Springer, Berlin, Heidelberg (Aug 2013). https://doi.org/10.1007/978-3-642-40041-4_3
25. Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR TCHES* **2018**(1), 238–268 (2018). <https://doi.org/10.13154/tches.v2018.i1.238-268>, <https://tches.iacr.org/index.php/TCHES/article/view/839>
26. Fenzi, G., Knabenhans, C., Nguyen, N.K., Pham, D.T.: Lova: Lattice-based folding scheme from unstructured lattices. In: Chung, K.M., Sasaki, Y. (eds.) ASIACRYPT 2024, Part IV. LNCS, vol. 15487, pp. 303–326. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0894-2_10
27. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) CRYPTO’86. LNCS, vol. 263, pp. 186–194. Springer, Berlin, Heidelberg (Aug 1987). https://doi.org/10.1007/3-540-47721-7_12
28. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: Permutations over Lagrange-bases for oecumenical non-interactive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953 (2019), <https://eprint.iacr.org/2019/953>
29. Garreta, A., Lipmaa, H., Luhaäär, U., Osadnik, M.: Cyclo: Lightweight lattice-based folding via partial range checks. Eurocrypt 2026, to appear (2026), <https://eprint.iacr.org/2026/359>
30. Gentry, C., Halevi, S., Lyubashevsky, V.: Practical non-interactive publicly verifiable secret sharing with thousands of parties. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part I. LNCS, vol. 13275, pp. 458–487. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-06944-4_16
31. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part II. LNCS, vol. 9666, pp. 305–326. Springer, Berlin, Heidelberg (May 2016). https://doi.org/10.1007/978-3-662-49896-5_11
32. Klooss, M., Lai, R.W.F., Nguyen, N.K., Osadnik, M., Tucci, L.: RoKoko: Lattice-based succinct arguments, a committed refinement. Cryptology ePrint Archive, Paper 2026/575 (2026), <https://eprint.iacr.org/2026/575>
33. Kuriyama, S., Lai, R.W.F., Osadnik, M., Tucci, L.: SALSAA – sumcheck-aided lattice-based succinct arguments and applications. Cryptology ePrint Archive, Report 2025/2124 (2025), <https://eprint.iacr.org/2025/2124>
34. Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. *DCC* **75**(3), 565–599 (2015). <https://doi.org/10.1007/s10623-014-9938-4>
35. Lyubashevsky, V.: Lattice signatures without trapdoors. In: Pointcheval, D., Johansson, T. (eds.) EUROCRYPT 2012. LNCS, vol. 7237, pp. 738–755. Springer, Berlin, Heidelberg (Apr 2012). https://doi.org/10.1007/978-3-642-29011-4_43

36. Lyubashevsky, V., Nguyen, N.K., Plançon, M.: Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part II. LNCS, vol. 13508, pp. 71–101. Springer, Cham (Aug 2022). https://doi.org/10.1007/978-3-031-15979-4_3
37. Lyubashevsky, V., Nguyen, N.K., Plançon, M., Seiler, G.: Shorter lattice-based group signatures via “almost free” encryption and other optimizations. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part IV. LNCS, vol. 13093, pp. 218–248. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92068-5_8
38. Lyubashevsky, V., Nguyen, N.K., Seiler, G.: Shorter lattice-based zero-knowledge proofs via one-time commitments. In: Garay, J. (ed.) PKC 2021, Part I. LNCS, vol. 12710, pp. 215–241. Springer, Cham (May 2021). https://doi.org/10.1007/978-3-030-75245-3_9
39. Lyubashevsky, V., Seiler, G.: Short, invertible elements in partially splitting cyclotomic rings and applications to lattice-based zero-knowledge proofs. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018, Part I. LNCS, vol. 10820, pp. 204–224. Springer, Cham (Apr / May 2018). https://doi.org/10.1007/978-3-319-78381-9_8
40. Lyubashevsky, V., Seiler, G., Steuer, P.: The LaZer library: Lattice-based zero knowledge and succinct proofs for quantum-safe privacy. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) ACM CCS 2024. pp. 3125–3137. ACM Press (Oct 2024). <https://doi.org/10.1145/3658644.3690330>
41. Merkle, R.C.: One way hash functions and des. In: Brassard, G. (ed.) CRYPTO 1989. LNCS, vol. 435, pp. 428–446. Springer, New York, NY (1990). https://doi.org/10.1007/0-387-34805-0_40
42. Nguyen, N.K., Seiler, G.: Greyhound: Fast polynomial commitments from lattices. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part X. LNCS, vol. 14929, pp. 243–275. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68403-6_8
43. Nguyen, W., Setty, S.: Neo: Lattice-based folding scheme for CCS over small fields and pay-per-bit commitments. Cryptology ePrint Archive, Report 2025/294 (2025), <https://eprint.iacr.org/2025/294>
44. Pino, R.D., Katsumata, S., Niot, G., Reichle, M., Takemure, K.: Unmasking tracoon: A lattice-based threshold signature with an efficient identifiable abort protocol. In: Kalai, Y.T., Kamara, S.F. (eds.) CRYPTO 2025, Part VI. pp. 423–456. LNCS, Springer (2025). https://doi.org/10.1007/978-3-032-01887-8_14, https://doi.org/10.1007/978-3-032-01887-8_14
45. Valiant, P.: Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In: Canetti, R. (ed.) TCC 2008. LNCS, vol. 4948, pp. 1–18. Springer, Berlin, Heidelberg (Mar 2008). https://doi.org/10.1007/978-3-540-78524-8_1
46. Wahby, R.S., Tzialla, I., Shelat, A., Thaler, J., Walfish, M.: Doubly-efficient zkSNARKs without trusted setup. In: 2018 IEEE Symposium on Security and Privacy. pp. 926–943. IEEE Computer Society Press (May 2018). <https://doi.org/10.1109/SP.2018.00060>

Appendix

A Deferred Preliminaries

We consider the norm of elements in $\mathcal{R}$ to be $\|a\|_p = |a|$ if $a \in \mathbb{Z}$ and $\|a\|_p = \sqrt[p]{\sum |a_i|^p}$ if $a = \sum a_i X^i \in \mathcal{R}$, for $p > 1$. We extend the notation to vectors $\|\mathbf{a}\|_p = \sqrt[p]{\sum_i \|\mathbf{a}_i\|_p^p}$. We remove the subscript of $\|\cdot\|_p$ when $p = 2$. In the quotient ring, the norm of an element in $\mathcal{R}_q$ is the norm of its unique representative $\mathcal{R}$ with coefficients in $[-\frac{q-1}{2}, \frac{q-1}{2}]$.

In power-of-two cyclotomic rings, working over subrings $\mathcal{S}$ of $\mathcal{R}$ is possible, as detailed in [37, Section 2.8.]. Namely, if $\mathcal{S} = \mathbb{Z}[X]/(X^{d'} + 1)$ a subring of $\mathcal{R}$, for $d'|d$, then, there is a norm-preserving bijection $\Phi : \mathcal{R} \rightarrow \mathcal{S}^{d/d'}$, using the embedding of $\mathcal{S}$ in $\mathcal{R}$ via $X \rightarrow X^{d/d'}$. This bijection can be naturally extended to vectors over $\mathcal{S}$ and $\mathcal{R}$ and over their quotients mod q . Using this embedding, one could translate the multiplication by a ring element $a \in \mathcal{R}$ as a linear map over $\mathcal{S}$, described by the so-called *rotation matrix* $\text{Rot}(a) \in \mathcal{S}^{d/d' \times d/d'}$ as $\Phi(ab) = \text{Rot}_a \cdot \Phi(b) \in \mathcal{S}^{d/d'}$. The rotation matrix $\text{Rot}(a)$ is defined as follows

$$\text{Rot}(a) := (\Phi(a) \Phi(aX) \dots \Phi(aX^{d/d'-1})).$$

We recall the result of [39] which says that short elements in $\mathcal{R}_q$ are invertible.

Lemma 1 (Cor.1.2, [39]). *Let $q \equiv 5 \pmod{8}$ be a prime. Then, any $f \in \mathcal{R}_q$ which satisfies either $0 \leq \|f\|_\infty < \frac{1}{2}q^{1/2}$ or $0 < \|f\| < q^{1/2}$ is invertible in $\mathcal{R}_q$.*

A.1 Rejection Sampling and the Gaussian Distribution

In lattice-based zero-knowledge proofs, the prover may want to produce $\mathbf{z}$ whose distribution is independent of a witness vector $\mathbf{v}$, so the verifier would not obtain any secret information. The achievement of this independence is done by *rejection sampling* [35] to ensure a target distribution. We use Gaussian distributions to sample the *mask* $\mathbf{y}$ in our proofs of knowledge.

Definition 6. *The discrete Gaussian distribution over $\mathcal{R}^n$ centered at $\mathbf{v} \in \mathcal{R}^n$, with standard deviation $\sigma > 0$ is defined by $D_{\mathbf{v}, \sigma}^n(x) := \frac{\exp(-\frac{\|\mathbf{x} - \mathbf{v}\|^2}{2\sigma^2})}{\sum_{\mathbf{x}' \in \mathcal{R}^n} \exp(-\frac{\|\mathbf{x}'\|^2}{2\sigma^2})}$. When it is centered around $\mathbf{0} \in \mathcal{R}^n$, we simply write $D_\sigma^n = D_{\mathbf{0}, \sigma}^n$.*

We also use the following Gaussian tail bound, following from [6, Lem 1.5(i)] and adapted in [5, Lem 2.5].

Lemma 2. $\Pr_{\mathbf{z} \leftarrow D_\sigma^n} [\|\mathbf{z}\| < \sigma\sqrt{2nd}] > 1 - 2^{-nd/2 \cdot \log(e/2)} > 1 - 2^{-nd/8}$.

Lemma 3 ([35, 38]). *Let $V \subseteq \mathcal{R}^n$ be a set of vectors with norm at most B and $\rho : V \rightarrow [0, 1]$ be a probability distribution. Let $\sigma = \gamma B$ a standard deviation, for some constant $\gamma > 0$.*

- 1 Let $M := \exp(14/\gamma + 1/(2\gamma^2))$. Sample $\mathbf{v} \leftarrow \rho$ and $\mathbf{y} \leftarrow D_\sigma^n$ and set $\mathbf{z} = \mathbf{y} + \mathbf{v}$. Run $b \leftarrow \text{Rej}_1(\mathbf{z}, \mathbf{v}, \sigma)$ from Fig. 2. Then the probability that $b = 0$ is at least $(1 - 2^{-128})/M$ and the distribution of $(\mathbf{v}, \mathbf{z})$, conditioned on $b = 0$, is within statistical distance of 2^{-128} of $\rho \times D_\sigma^n$.
- 2 Let $M := \exp(1/(2\gamma^2))$. Sample $\mathbf{v} \leftarrow \rho$ and $\mathbf{y} \leftarrow D_\sigma^n$ and set $\mathbf{z} = \mathbf{y} + \mathbf{v}$. Run $b \leftarrow \text{Rej}_2(\mathbf{z}, \mathbf{v}, \sigma)$ from Fig. 2. Then the probability that $b = 0$ is at least $1/(2M)$ and the distribution of $(\mathbf{v}, \mathbf{z})$, conditioned on $b = 0$, is identical to the one obtained as follows: sample $\mathbf{s} \leftarrow \rho$, $\mathbf{z} \leftarrow D_\sigma^n$, conditioned on $\langle \mathbf{z}, \mathbf{v} \rangle \geq 0$ output $(\mathbf{v}, \mathbf{z})$ with probability $1/M$.

<pre> Rej₁(z, v, σ) 00 $u \leftarrow [0, 1)$ 01 if $u > \frac{1}{M} \cdot \exp\left(\frac{-2\langle \mathbf{z}, \mathbf{v} \rangle + \ \mathbf{v}\ ^2}{2\sigma^2}\right)$ 02 return 1 (i.e. <i>reject</i>) 03 else 04 return 0 (i.e. <i>accept</i>) </pre>	<pre> Rej₂(z, v, σ) 05 if $\langle \mathbf{z}, \mathbf{v} \rangle < 0$ 06 return 1 (i.e. <i>reject</i>) 07 $u \leftarrow [0, 1)$ 08 if $u > \frac{1}{M} \cdot \exp\left(\frac{-2\langle \mathbf{z}, \mathbf{v} \rangle + \ \mathbf{v}\ ^2}{2\sigma^2}\right)$ 09 return 1 (i.e. <i>reject</i>) 10 else 11 return 0 (i.e. <i>accept</i>) </pre>
--	--

Fig. 2. Two rejection sampling algorithms: [35] (left) and [38] (right).

A.2 The Johnson-Lindenstrauss Lemmas

These results use Gaussian tail bounds derived under the heuristic of [30] of the projection matrices with entries from the binomial distribution Bin_k , having instead normally distributed entries.

Theorem 3. *Under the heuristic substitution of Bin_k with the normal distribution of standard deviation $\sqrt{\frac{k}{2}}$, from [30, Cor 3.2], for any $k \in \{1, 2\}$, we have for $\mathbf{w} \in \mathbb{Z}^n$,⁷*

$$\Pr_{\mathbf{r} \leftarrow \text{Bin}_1^n} [|\langle \mathbf{r}, \mathbf{w} \rangle| > 9.75 \|\mathbf{w}\|] < \frac{1}{2^{141}},$$

$$\Pr_{\mathbf{r} \leftarrow \text{Bin}_1^n} [|\langle \mathbf{r}, \mathbf{w} \rangle| \leq 0.74 \|\mathbf{w}\|] < \frac{1}{\sqrt{2}}.$$

We recall the Johnson-Lindenstrauss lemma from [13] that relates the infinity norm of the projection with the ℓ_2 norm of the witness.

Theorem 4. *For every vector $\mathbf{w} \in [\pm q/2]^n$ with $\|\mathbf{w}\|_2 \geq b$, for some bound $b \leq q/91$, we have*

$$\Pr_{\mathbf{\Pi} \leftarrow \text{Bin}_1^{256 \times n}} [\|\mathbf{\Pi} \mathbf{w} \bmod q\|_\infty \leq 0.74b] < \frac{1}{2^{128}}.$$

Here we introduce a new Johnson-Lindenstrauss lemma that relates the ℓ_∞ norm of the masked projection with the ℓ_2 norm of the witness. It follows from [13, 36].

Theorem 5. *For every vector $\mathbf{w} \in [\pm q/2]^n$ with $\|\mathbf{w}\|_2 \geq b$, for some bound $b \leq q/165$, we have*

$$\Pr_{\mathbf{\Pi} \leftarrow \text{Bin}_1^{256 \times n}} [\|\mathbf{\Pi} \mathbf{w} + \mathbf{y} \bmod q\|_\infty \leq \frac{1}{2} 0.67b] < \frac{1}{2^{128}}.$$

The theorem follows the one of [36, Lem 2.9.], which requires an auxiliary lemma involving the infinity norm of the projection $\mathbf{\Pi} \mathbf{w}$, for $\mathbf{\Pi} \leftarrow \text{Bin}_2$. The latter lemma follows [13, Thm.6], adapted to $\text{Bin}_2^{256 \times n}$ distribution, aiming for a smaller failure probability, $1/2^{256}$. Thanks to the heuristic of [30, Cor. 3.2], we can assume that the entries of $\mathbf{\Pi}$ follow the standard normal distribution, which might lead to a slightly different application of the Berry-Esseen theorem in the proof of [13, Thm.6].

A.3 Particular Constraints

Here we recall how binary proofs or exact norm proofs can be given, using lattice techniques from previous work [11, 36].

⁷ The bounds were computed via a python command, see
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.chi2.html>

Binary proof. For proving a vector $\tilde{\mathbf{x}} \in \mathcal{R}_q^m$ is made only of ring elements with binary coefficients, or equivalently, the coefficients of any entry of this vector are binary vectors, we make use of the observation [36, Lem 5.2] saying that an integer vector $\mathbf{x} \in \mathbb{Z}^{md}$ is binary if and only if $\langle \mathbf{x}, \mathbf{x} - \mathbf{1}_{md} \rangle = 0$ over the integers, where $\mathbf{1}_n$ is the integer vector with all entries equal to 1. This translates into showing that the corresponding polynomial vector $\tilde{\mathbf{x}}$ over $\mathcal{R}$ satisfies

$$\text{ct}(\langle \sigma_{-1}(\tilde{\mathbf{x}} - \mathbf{1}), \tilde{\mathbf{x}} \rangle) = 0. \quad (20)$$

For proving this, we need to add an auxiliary witness vector, $\sigma_{-1}(\tilde{\mathbf{x}})$. However, one could prove Equation (20) only mod q . In order to prove this holds over $\mathbb{Z}$, we would need to also prove that the vector $\tilde{\mathbf{x}}$ is small, i.e. $\|\tilde{\mathbf{x}}\| \leq \sqrt{md}$, by projecting it as before, and thus prove its norm, up to some slack factor α . As the left hand side of Equation (20) is $|\langle \mathbf{x}, \mathbf{x} - \mathbf{1}_{md} \rangle| \leq 2\|\mathbf{x}\|^2$, for any $\mathbf{x} \in \mathbb{Z}^{md}$ integer vector, by letting $2 \cdot \alpha^2 \cdot md < q$, we guarantee that Equation (20) holds over the integers.

For a binary vector polynomial input $\mathbf{v} \in \mathcal{R}^n$, **bin-check** works as follows:

- append $\sigma_{-1}(\mathbf{v})$ to $\mathbf{v}$,
- include constraints on correctness of $\sigma_{-1}(\mathbf{v})$,
- include $\text{ct}(\langle \mathbf{v}, \sigma_{-1}(\mathbf{v} - \mathbf{1}) \rangle) = 0$,
- include approximate norm proof for $\|\mathbf{v}\| \leq \sqrt{d \cdot \dim(\mathbf{v})}$.

Automorphism proof. As particular constraints, such as binary or exact norm proofs, involve adding new witness vectors when applying the automorphism σ_{-1} , one needs to prove their correctness. More precisely, as in [1, 11], we have to prove that the k -th coefficient of some polynomial $a \in \mathcal{R}$ matches the $(d - k)$ -th coefficient of $\tilde{a} := \sigma_{-1}(a)$, up to a sign, as in

$$\text{ct}(X^{d-k}a - X^k\tilde{a}) = 0. \quad (21)$$

B Deferred proofs

B.1 On the Outer Commitment Layer

Here we show that applying the outer commitment layer preserves the knowledge soundness and zero-knowledge. For achieving zero-knowledge, one needs to use the hiding version of the outer commitment.

Lemma 4. *Let Π be a knowledge sound proof system and an extractor $\mathcal{E}$ that extracts with knowledge error κ a valid witness for Π . Let Π' be a derived protocol from Π by applying the outer commitment layer with hiding, i.e. any prover's message is postponed to the last message and sent instead in a committed form. Then there exists an extractor $\mathcal{E}'$ that either produces a valid witness for Π' or breaks the binding of the outer commitment layer.*

Proof. This extractor $\mathcal{E}'$ will use as subroutine an extractor $\mathcal{E}$ for the protocol Π . Indeed, let $\mathcal{E}'$ run the prover of Π' and look at his last message. By construction, the last message contains the openings of the commitments the prover sent earlier in Π' . The binding property of the outer commitment layer ensures that these openings are exactly the prover's messages in Π . Then $\mathcal{E}$ can reconstruct the transcript of Π . He uses $\mathcal{E}$ to extract a valid witness for Π , which, by construction, turns out to be a valid witness for Π' .

Lemma 5. *Let Π be a zero-knowledge proof system and Π' be a derived protocol from Π by applying the outer commitment layer with hiding, i.e. any prover's message is postponed to the last message and sent instead in a committed form. Then Π' also satisfies the zero-knowledge property.*

Proof. We extend in a straightforward way the simulator of Π to the additional commitment layer involved in Π' . Namely, we construct a simulator $\mathcal{S}'$ for the protocol Π' by using as subroutine a simulator $\mathcal{S}$ for the protocol Π . As $\mathcal{S}$ simulates a valid transcript for Π , indistinguishable from the real distribution, thanks to zero-knowledge, $\mathcal{S}'$ simply takes all the messages from this transcript and creates commitments of them, in order to create a valid transcript for the protocol Π' . By the hiding of the commitment layer, these commitments look indistinguishable from the ones of the messages from the real transcript of Π .

B.2 Security of LNP-Lite

Completeness error. In the first projection step, the threshold N is set (Section D) such that each of the four masked projections has less than N carries with probability $p_1 := 0.99$. Note that the projections are not independent since they are derived from the same projection matrix. Then, by union bound, we can upper bound the probability that one of them is rejected by $4(1 - p_1)$, which leads to the probability that they all are accepted to be at least $1 - 4(1 - p_1)$. In the second projection step, by rejection sampling, $\mathbf{z}$ follows a distribution close to D_σ and accepted with probability $p_2 := \exp(14/\gamma + 1/(2\gamma^2))$ (Lemma 3 (1)), where $\sigma = \gamma\sqrt{337\sqrt{4N}}$. By the Gaussian tail bound (Lemma 2), the verifier checks $\|\mathbf{z}\| \leq \sigma\sqrt{2 \cdot 256}$ with probability at least $1 - 2^{-128}$. In the last step, the prover obtains by rejection sampling $\mathbf{z}_i$ following a close distribution to D_{σ_i} and passes both the rejection sampling algorithms with probability $1/\exp(14/\gamma_1 + 1/(2\gamma_1^2)) \cdot 1/(2\exp(1/(2\gamma_2^2)))$, where $\sigma_1 = \gamma_2 \cdot T \cdot \|(\tilde{\mathbf{s}}, \tilde{\mathbf{v}})\|$ and $\sigma_2 = \gamma_2 \cdot T \cdot \|(\mathbf{r}_s, \mathbf{r}_v)\|$. Again, by tail bound (Lemma 2), for $n_i, n_s + n_v \geq 128 \cdot 8/d = 4$, the verifier checks Equations (13) to (15) with probability at least $1 - 2^{-128}$ each. Then the honest prover convinces the verifier with probability

$$\simeq (1 - 4(1 - p_1))p_2 \cdot 1/(2\exp(14/\gamma_1 + 1/(2\gamma_1^2)) + 1/(2\gamma_2^2)).$$

Zero-knowledge We follow the lines of [36, Thm 4.2]. We recall the prover messages in LNP-Lite (Section 3.3): $\mathbf{t}_s, \mathbf{t}_v, \{v_i^{(0)}\}_i, \mathbf{z}, \{h_i\}_i, \mathbf{w}_s, \mathbf{w}_v, u, v, \mathbf{z}_1, \mathbf{z}_2$. We show that we can simulate a non-aborting transcript between honest prover and verifier through a series of simulators.

- The first simulator $\mathcal{S}_0$ knows the witness vector $\tilde{\mathbf{s}} = (\mathbf{s}_i)_{i \in [6]}$ and honestly computes its commitment $\mathbf{t}_s$ under randomness $\mathbf{r}_s$ (sampled from the uniform binary distribution) and the masked projections of $\mathbf{s}_i, i \in [4]$, to derive the high-order values and the low-order bits. Then he is able to compute the second commitment $\mathbf{t}_v$ to the high-order values (and their σ_{-1} 's), $\tilde{\mathbf{v}}$, under randomness $\mathbf{r}_v$ (sampled from the uniform binary distribution). The simulator samples $\mathbf{z}_1 \leftarrow D_{\sigma_1}$, and $h_i \leftarrow \{x \in \mathcal{R}_q \mid \text{ct}(x) = 0\}$, for $i \in [\lambda]$. Given verifier challenges, the simulator can also compute $u' \in \mathcal{R}_q$, (one of the garbage terms needed for the final quadratic check, see Equation (5)) and its commitment $u := \langle \varphi_{\text{rand}}, \mathbf{r}_s \rangle + u'$. Given a challenge $c \leftarrow \mathcal{C}$, he also samples $\mathbf{z}_2 := (\mathbf{z}_2^s, \mathbf{z}_2^v) \leftarrow D_{\sigma_2}$ such that $\langle \mathbf{z}_2, c(\mathbf{r}_s, \mathbf{r}_v) \rangle \geq 0$ and sets $\mathbf{w}_s := \mathbf{A}_1 \mathbf{z}_1^s + \mathbf{A}_1' \mathbf{z}_2^s - \text{ct}_{\mathbf{t}_s} \in \mathcal{R}_q^{k_s}, \mathbf{w}_v := \mathbf{A}_2 \mathbf{z}_1^v + \mathbf{A}_2' \mathbf{z}_2^v - \text{ct}_{\mathbf{t}_v} \in \mathcal{R}_q^{k_v}$. Then, by Lemma 3, the transcript output of $\mathcal{S}_0$ is statistically close to the one of the non-aborting protocol.
- The second simulator $\mathcal{S}_1$ still knows $\tilde{\mathbf{s}}$ (and thus $\tilde{\mathbf{v}}$) and runs identically to $\mathcal{S}_0$, but it computes the commitments $(\mathbf{t}_s, u, \mathbf{t}_v)$ in the following way: it samples $(\mathbf{u}_1, \mathbf{u}_2) \leftarrow \mathcal{R}_q^{k_s+1} \times \mathcal{R}_q^{k_v}$ and sets

$$\begin{pmatrix} \mathbf{t}_s \\ u \end{pmatrix} := \mathbf{u}_1 + \begin{pmatrix} \mathbf{A}_1 \tilde{\mathbf{s}} \\ u' \end{pmatrix} \in \mathcal{R}_q^{k_s+1} \quad (22)$$

$$\mathbf{t}_v := \mathbf{u}_2 + \mathbf{A}_2 \tilde{\mathbf{v}} \in \mathcal{R}_q^{k_v}. \quad (23)$$

Note that in the real transcript and in the output of $\mathcal{S}_0$

$$\begin{pmatrix} \mathbf{t}_s \\ u \end{pmatrix} = \begin{pmatrix} \mathbf{A}_1' \\ \varphi_{\text{rand}}^T \end{pmatrix} \mathbf{r}_s + \begin{pmatrix} \mathbf{A}_1 \tilde{\mathbf{s}} \\ u' \end{pmatrix} \in \mathcal{R}_q^{k_s+1}$$

$$\mathbf{t}_v = \mathbf{A}_2' \mathbf{r}_v + \mathbf{A}_2 \tilde{\mathbf{v}} \in \mathcal{R}_q^{k_v},$$

and in both cases, the sign of $\langle \mathbf{z}_2, c(\mathbf{r}_s, \mathbf{r}_v) \rangle$ is known. This looks like the Extended M-LWE problem but with two secrets, $\mathbf{r}_s, \mathbf{r}_v$, where the sign of the inner product of their (scaled) concatenation with $\mathbf{z}_2$ is leaked. We claim that if a PPT adversary $\mathcal{A}$ can distinguish between the outputs of $\mathcal{S}_1$ and $\mathcal{S}_0$ with probability ϵ , then there is a PPT adversary $\mathcal{B}$ that can break the two corresponding Extended M-LWE problems with probability at least $\epsilon/4$. Therefore, if the two-Extended M-LWE problems are hard, then the outputs of the simulators $\mathcal{S}_0$ and $\mathcal{S}_1$ should be indistinguishable. Indeed, assume that $\mathcal{B}$ is given $(\mathbf{C}_1, \mathbf{u}_1, c, \mathbf{z}_2^s, b_1)$ and $(\mathbf{C}_2, \mathbf{u}_2, c, \mathbf{z}_2^v, b_2)$

$$\mathbf{C}_1 := \begin{pmatrix} \mathbf{A}_1' \\ \varphi_{\text{rand}}^T \end{pmatrix}, \mathbf{C}_2 = \mathbf{A}_2'.$$

Then $\mathcal{B}$ computes $(\mathbf{t}_s, u, \mathbf{t}_v)$ as in Eqs. (22) and (23) and simulates the rest of the transcript identically to that above and sends it to $\mathcal{A}$. Let us assume that $b_1 = b_2 = 1$, so then $\langle \mathbf{z}_2^s, \mathbf{c}\mathbf{r}_s \rangle, \langle \mathbf{z}_2^v, \mathbf{c}\mathbf{r}_v \rangle \geq 0$. Summing up, we get $\langle \mathbf{z}_2, \mathbf{c}(\mathbf{r}_s, \mathbf{r}_v) \rangle \geq 0$, for $\mathbf{z}_2 = (\mathbf{z}_2^s, \mathbf{z}_2^v)$. If $\mathbf{u}_1 = \begin{pmatrix} \mathbf{A}'_1 \\ \varphi_{rand}^T \end{pmatrix} \mathbf{r}_s$ and $\mathbf{u}_2 = \mathbf{A}'_2 \mathbf{r}_v$, then the output of $\mathcal{B}$ is exactly the one of $\mathcal{S}_0$. If $\mathbf{u}_1$ and $\mathbf{u}_2$ are both uniform, then, the output of $\mathcal{B}$ is exactly the one of $\mathcal{S}_1$. Then, conditioned on $b_1 = b_2 = 1$, the adversary $\mathcal{B}$ solves the Extended-MLWE with probability ϵ . But since b_1, b_2 are both 1 with probability at least $1/4$, the claim follows.

- The third simulator $\mathcal{S}_2$ does not know any information about $\tilde{\mathbf{s}}$ and runs identically to $\mathcal{S}_1$, except that it sets $(\mathbf{t}_s, u, \mathbf{t}_v) := \mathbf{u} \leftarrow \mathcal{R}_q^{k_s+k_v+1}$. Then the distribution of the output of $\mathcal{S}_2$ is exactly the same as of $\mathcal{S}_1$. We get the conclusion by using hybrid argument.

Soundness Here we present the proof of Theorem 2.

Proof. As the outer commitment preserves knowledge soundness (Lemma 4), it is enough to focus on proving knowledge soundness of our zero-knowledge layer without outer commitments. Recall the output vectors of **LaBRADOS**, $(\mathbf{s}_1, \dots, \mathbf{s}_4)$, which satisfy the checks of **LaBRADOS** and the norm bounds $\beta_i, i \in [4]$. Let us collect the full-constraints of the **LaBRADOS** checks in $\mathcal{F}$ and the constant-term ones in $\mathcal{F}'$. Note that **bin-check** for proving $\mathbf{s}_4$ being binary considers $\mathbf{s}_5 = \sigma_{-1}(\mathbf{s}_4)$ and the corresponding constant-term constraints, already collected in $\mathcal{F}'$. Keep in mind that an approximate proof for $\mathbf{s}_5$ is not needed. All of these output vectors, together with the additional vector $\mathbf{s}_6$ made by the masks, form the witness $\tilde{\mathbf{s}}$ of our zero-knowledge layer and are all committed by hiding the Ajtai commitment, with the randomness vector $\mathbf{r}_s$. For simplicity, we extend the domains of the constraints from $\mathcal{F}$ and $\mathcal{F}'$ to be satisfied by $\tilde{\mathbf{s}}$, although $\mathbf{s}_6$ is not involved. Formally, our zero-knowledge layer gives a proof-of-knowledge for the following relation

$$R_{\text{LNP}} = \left\{ \begin{array}{l} ((\mathbf{t}_s, \mathcal{F}, \mathcal{F}', (\beta_i)_i), \\ (\tilde{\mathbf{s}}, \mathbf{r}_s)) \end{array} \left| \begin{array}{l} \mathbf{t}_s = \mathbf{A}_1 \tilde{\mathbf{s}} + \mathbf{A}'_1 \mathbf{r}_s \\ f(\tilde{\mathbf{s}}) = 0, \forall f \in \mathcal{F} \\ \text{ct}(f'(\tilde{\mathbf{s}})) = 0, \forall f' \in \mathcal{F}' \\ \|\mathbf{s}_1\| \leq \bar{\gamma}\beta_1, \dots, \|\mathbf{s}_4\| \leq \bar{\gamma}\beta_4 \end{array} \right. \right\}.$$

for $\bar{\gamma} := \frac{2 \cdot 9.75}{0.74}$. Recall that in the first projection step, each of the vectors $\mathbf{s}_i$, for $i \in [4]$ receives a masked projection, using random projection matrices $\mathbf{\Pi}_i$, all coming from the same $\mathbf{\Pi}$ of the verifier, and the uniform masks $\mathbf{u}_i$, part of the vector $\mathbf{s}_6$. Decomposing these projections, we get the high-order values and low-order bits:

$$\mathbf{\Pi}_i \mathbf{s}_i + \mathbf{u}_i = 2^{k_i}(\mathbf{w}_i - \mathbf{t}_i) + \mathbf{v}_i^{(0)}, \quad \forall i \in [4], \quad (24)$$

where $k_i = \lceil \log(9.75\beta_i) \rceil$. All low-order bits, $\mathbf{v}_i^{(0)}$, could be sent in the clear, as they are uniform. All high-order values, represented by the binary vectors $\mathbf{w}_i$ and $\mathbf{t}_i$, are collected in the vector $\tilde{\mathbf{v}}_1 \in \mathcal{R}^8$. These masked projections are accepted if they contain less than N nonzero high-order values, i.e. $\|(\mathbf{w}_i, \mathbf{t}_i)\| \leq \sqrt{N}$, each with probability p_1 . Then, (with probability $1 - 4(1 - p_1)$), we can construct a new witness vector $\tilde{\mathbf{v}}_1$, for which we prove binary and $\|\tilde{\mathbf{v}}_1\| \leq \sqrt{4N}$. For this, we also consider $\tilde{\mathbf{v}}_2 = \sigma_{-1}(\tilde{\mathbf{v}}_1)$ and, thus a hiding Ajtai commitment $\mathbf{t}_v$ to $\tilde{\mathbf{v}} = (\tilde{\mathbf{v}}_1, \tilde{\mathbf{v}}_2)$ is provided, under randomness $\mathbf{r}_v$. The projections from Equation (24) get translated into new constant-term constraints to be proven to be satisfied by $\tilde{\mathbf{s}}$ and $\tilde{\mathbf{v}}$, and we collect them all into $\mathcal{F}''$. Note that starting from the second step of projection, the zero-knowledge layer follows exactly [36, Sec. 5], so we can treat the following steps together as a subprotocol. Therefore, formally written, the first projection step could be viewed as reducing R_{LNP} to the following relation

$$R'_{\text{LNP}} = \left\{ \begin{array}{l} ((\mathbf{t}_s, \mathbf{t}_v, (\mathbf{v}_i^{(0)})_i), \\ \mathcal{F}, \mathcal{F}', (\beta_i)_i, \\ N, \mathbf{\Pi}), \\ (\tilde{\mathbf{s}}, \tilde{\mathbf{v}}, \mathbf{r}_s, \mathbf{r}_v)) \end{array} \left| \begin{array}{l} \mathbf{t}_s = \mathbf{A}_1 \tilde{\mathbf{s}} + \mathbf{A}'_1 \mathbf{r}_s \\ \mathbf{t}_v = \mathbf{A}_2 \tilde{\mathbf{v}} + \mathbf{A}'_2 \mathbf{r}_v \\ f(\tilde{\mathbf{s}}) = 0, \forall f \in \mathcal{F} \\ \text{ct}(f'(\tilde{\mathbf{s}})) = 0, \forall f' \in \mathcal{F}' \\ \text{ct}(f''(\tilde{\mathbf{s}}, \tilde{\mathbf{v}})) = 0, \forall f'' \in \mathcal{F}'' \\ \text{bin-check}(\tilde{\mathbf{v}}) \\ \|\tilde{\mathbf{v}}_1\| \leq \gamma' \sqrt{4N} \end{array} \right. \right\}.$$

for $\gamma' := 2\sqrt{2 \cdot 256/26}\gamma\sqrt{337}$, for a repetition rate γ .

Our extractor $\mathcal{E}$ for the zero-knowledge protocol uses as a subroutine the extractor $\mathcal{E}'$ of the subprotocol. Therefore, we can rely on the knowledge soundness of the latter to extract a witness $\bar{\mathbf{s}}, \bar{\mathbf{v}}, \bar{\mathbf{r}}_s, \bar{\mathbf{r}}_v$ that satisfies

$$\begin{aligned} \mathbf{t}_s &= \mathbf{A}_1 \bar{\mathbf{s}} + \mathbf{A}'_1 \bar{\mathbf{r}}_s \\ \mathbf{t}_v &= \mathbf{A}_2 \bar{\mathbf{v}} + \mathbf{A}'_2 \bar{\mathbf{r}}_v \\ f(\bar{\mathbf{s}}) &= 0, \forall f \in \mathcal{F} \\ \text{ct}(f'(\bar{\mathbf{s}})) &= 0, \forall f' \in \mathcal{F}' \\ \text{ct}(f''(\bar{\mathbf{s}}, \bar{\mathbf{v}})) &= 0, \forall f'' \in \mathcal{F}'' \\ \text{bin-check}(\bar{\mathbf{v}}_1) & \end{aligned} \tag{25}$$

except with probability $\kappa' := 1/2^{128} + (1/q)^{d/2} + (1/q)^\lambda + 2/|\mathcal{C}|$ (see [36, Prop.5.1]). Moreover, if the value of any of $\bar{\mathbf{s}}, \bar{\mathbf{v}}, \bar{\mathbf{r}}_s, \bar{\mathbf{r}}_v$ was dependent on the challenge $\mathbf{\Pi}$, then we would be able to use the knowledge extractor $\mathcal{E}'$ to break the binding of the commitments $\mathbf{t}_s$ and $\mathbf{t}_v$, i.e. the M-SIS problems for ℓ_2 norm bound $8T\sqrt{B_s^2 + B_r^2}$ and rank k_s and for ℓ_2 norm bound $8T\sqrt{B_v^2 + B_r^2}$ and rank k_v . Therefore, $\bar{\mathbf{s}}, \bar{\mathbf{v}}, \bar{\mathbf{r}}_s, \bar{\mathbf{r}}_v$ are unique and, in particular, do not depend on $\mathbf{\Pi}$. Thus, it suffices to compute the probability, over the randomness of $\mathbf{\Pi}$ that the extracted vectors do not satisfy the relation R_{LNP} . As the extracted vectors already satisfy the **LaBRADOS** checks from $\mathcal{F}$ and $\mathcal{F}'$, it suffices to compute the probability that one of the vectors $\bar{\mathbf{s}}_i$, for some $i \in [4]$, does not satisfy the norm bound, i.e. $\|\bar{\mathbf{s}}_i\| > \bar{\gamma}\beta_i$.

First, we argue that the extractor $\mathcal{E}'$ of the subprotocol extracts the vector $\bar{\mathbf{v}}_1 \in \mathcal{R}^8$ that is indeed binary. Note that this vector satisfies $\|\bar{\mathbf{v}}_1\| \leq 2\sqrt{2 \cdot 256/26}\gamma\sqrt{337}\sqrt{4N} = 2\sigma\sqrt{512/26}$. ([36, Prop 5.1.]). For this approximate norm bound, it was used $q \geq 41 \cdot 2048 \cdot 2\sigma\sqrt{512/26}$. As this vector $\bar{\mathbf{v}}_1$ satisfies **bin-check**, i.e. $\bar{\mathbf{v}}_2 = \sigma_{-1}(\bar{\mathbf{v}}_1)$ and $\langle \bar{\mathbf{v}}_1, \bar{\mathbf{v}}_2 \rangle - \langle \bar{\mathbf{v}}_1, \sigma_{-1}(\mathbf{1}) \rangle = 0 \pmod q$, we can upper-bound (the absolute value of) the left hand side of the latter equation, $2\|\bar{\mathbf{v}}_1\|^2 \leq 2 \cdot (2\sigma\sqrt{512/26})^2 \leq q$, so we are guaranteed that this equation actually holds over $\mathbb{Z}$. This proves that $\bar{\mathbf{v}}_1$, and, in particular, the vectors $\bar{\mathbf{w}}_i, \bar{\mathbf{t}}_i$ $\bar{\mathbf{v}}_1$ is made of, are indeed binary.

Second, we argue that the extracted vectors $\bar{\mathbf{s}}_i$ satisfy $\|\mathbf{\Pi}_i \bar{\mathbf{s}}_i\|_\infty \leq 2^{k_i+1}$. As the vectors extracted from $\mathcal{E}'$ satisfy Equation (25), they also satisfy Equation (24) written as follows

$$\mathbf{\Pi}_i \bar{\mathbf{s}}_i = 2^{k_i}(\bar{\mathbf{w}}_i - \bar{\mathbf{t}}_i) + \mathbf{v}_i^{(0)} - \bar{\mathbf{u}}_i \pmod q.$$

Recall that the vectors $\mathbf{v}_i^{(0)} \in [-2^{k_i-1}, 2^{k_i-1}-1]^{256}$ are public and the extractor $\mathcal{E}$ finds $\bar{\mathbf{u}}_i \in [-2^{k_i-1}, 2^{k_i-1}]^{256}$ from the length of its bit-decomposition. Moreover, since $\bar{\mathbf{w}}_i, \bar{\mathbf{t}}_i$ are binary, $\|\bar{\mathbf{w}}_i - \bar{\mathbf{t}}_i\|_\infty \leq 1$. Therefore, by the triangle inequality, the infinity norm of the right hand side could be at most $2^{k_i} + 2^{k_i-1} + 2^{k_i-1} = 2^{k_i+1}$. Moreover, q is set so big that no wraparound mod q is happening on this side. (Imagine q on 38 bits and **LaBRADOS**'s output norm bound β_i on 13-15 bits, so then $2^{k_i} \simeq 9.75\beta_i$ is on, say, 17-19 bits.)

Therefore, our extracted vectors $\bar{\mathbf{s}}_i$ satisfy $\|\mathbf{\Pi}_i \bar{\mathbf{s}}_i \pmod q\|_\infty \leq 2^{k_i+1}$, for any $i \in [4]$. By Theorem 4, if it happens that some of the extracted vectors $\bar{\mathbf{s}}_i$ are too long, i.e. they do not satisfy the norm bound $\|\bar{\mathbf{s}}_i\| \leq \frac{2^{k_i+1}}{0.74} \simeq \frac{2 \cdot 9.75}{0.74} \beta_i$, then $\|\mathbf{\Pi}_i \bar{\mathbf{s}}_i \pmod q\|_\infty \leq 2^{k_i+1}$ holds only with probability $1/2^{128}$, because of Theorem 4 (here we use $q > 91 \cdot 2^{k_i+1}/0.74$ for any $i \in [4]$). By union bound, the probability that at least one of the four norm checks fails is at most $4/2^{128}$.

Therefore, we obtain the knowledge error of the zero-knowledge layer as $\kappa := \kappa' + 4/2^{128}$. Thus, the extracted vectors $\bar{\mathbf{s}}_i$ satisfy $\|\bar{\mathbf{s}}_i\| \leq \frac{2 \cdot 9.75}{0.74} \beta_i$, and the **LaBRADOS** checks, collected in $\mathcal{F}, \mathcal{F}'$, unless with small probability. What is left to check is $\bar{\mathbf{s}}_4$ being binary: note that it already satisfies the **bin-check** function, so then, as long as $2\|\bar{\mathbf{s}}_4\|^2 \leq 2 \cdot d \cdot n_4 \cdot \left(\frac{2 \cdot 9.75}{0.74}\right)^2 \leq q$, the binary equation holds over $\mathbb{Z}$, which leads to the conclusion.

C The Fiat-Shamir Transformation

Public-coin interactive proofs (those where all the verifier's messages are randomly sampled values) can be turned *non-interactive* using the Fiat-Shamir transformation [27]. The security of the composition of LNP

(with outer commitments), as the starting protocol, with **LaBRADOR** was already studied in [44]. In our case, we have iterations of **LaBRADOS** (which is **LaBRADOR** with outer commitments), before going to **LNP-Lite**, a version of **LNP**, which is further fed into iterations of **LaBRADOS** and **LaBRADOR**. Thus, the security of our composition should follow almost the same lines as the security analyzed in [44], with the differences we detail below.

Completeness follows directly from the construction of the Fiat-Shamir transformation, whereas non-interactive zero-knowledge is derived by the fact that the composition itself is honest-verifier zero-knowledge [35]. Thus, we focus only on the aspect of knowledge soundness, looking at each kind of protocol, **LaBRADOS**, **LNP-Lite** and **LaBRADOR**.

[1] proved the knowledge soundness of **LaBRADOR** in the random oracle model (ROM) [7], using the *predicate special soundness* framework. This framework could be viewed as a natural extension of the special soundness that comes with a collection of *predicates*, i.e. properties satisfied (with an overwhelming probability) by the extracted openings and the challenges at each prover-verifier interaction. As the property of *predicate special soundness* is preserved when applying outer commitments on a proof system Π (as noted also by [44, Sec.B.6.1], due to the binding of the outer commitments, they never open to distinct values, a similar property to Lemma 4), then, due to [1, Thm 5.2.], we can heuristically rely on the soundness of **LaBRADOR** and **LaBRADOS** compiled via Fiat-Shamir. [1] also showed that predicate special soundness protocols are composed nicely and thus their composition can be soundly compiled via Fiat-Shamir (see [1, Thm 5.1]). Therefore, it suffices to analyze the soundness of our zero-knowledge layer in the discussed framework. The knowledge soundness of **LNP** in a non-interactive version was analyzed in [36, 44], where the latter explicitly used the *predicate special-soundness* machinery. Using our zero-knowledge protocol **LNP-Lite** (without outer commitments), we could derive its soundness proof as [44] cast in the *predicate special-soundness* framework. We recall that this protocol has, compared to **LNP**, an additional prover-verifier interaction, corresponding to the first projection step. In this step, note that the upper bound on the fraction of projection matrices (i.e. *failure density*), used in this step is at most $1/2^{128}$ (Theorem 4), since the projections of the invalid (i.e. of too big norm) extracted witness vectors to have a certain number of bits is at most $1/2^{128}$. Thus, in the analysis of [44, Lem.B.10], we inquire about an extra *failure density* value of $1/2^{128}$. As in **LaBRADOS**, the predicate special soundness of **LNP-Lite** easily extends when considering the outer commitments. Therefore, the whole composition of **LNP-Lite** with **LaBRADOS** and **LaBRADOR** has predicate special soundness, and thus shall be considered (heuristically) secure after applying the Fiat-Shamir transformation.

Following the observations from [44, Thm 5.1], the knowledge error κ of our construction in the ROM shall be adjusted for a multiplicative loss arising from the number of queries the adversary is allowed to make to the random oracle.

D Deferred Parameters

D.1 Parameters for **LaBRADOS**

The Modulus q . From Theorem 1, we need $\frac{9.75}{0.74}\beta_i \leq \frac{q}{91}$. For proving **bin-check** over $\mathbb{Z}$, (Section A.3), as all witness vectors of **LaBRADOS**, including $\mathbf{x}_{\text{bin}} \in \mathcal{R}^{n_{\text{bin}}}$, benefit from approximate range proofs with slack $c_{\text{Lab-new}} = \frac{9.75}{0.74}$, we set $2dn_{\text{bin}} \cdot c_{\text{Lab-new}}^2 \leq q$.

The rest of the parameters involved in **LaBRADOS** and **LaBRADOR** are chosen following the same rationale of [11, 13].

D.2 Parameters for **LNP-Lite**

We set the parameters for **LNP-Lite** described in Section 3.3. We use the challenge space $\mathcal{C}$ from Section 2.1 of ring elements of operator norm T .

The Dimension and Bound of the Additional Witness $\mathbf{s}_6$. Recall $\mathbf{s}_6 \in \mathcal{R}^{n_6}$ is made of the binary decompositions of the masks $\mathbf{u}, \mathbf{y}, \mathbf{g}$. As $\mathbf{y} \leftarrow D_\sigma$, with high probability $\|\mathbf{y}\|_\infty \leq 14\sigma$. Each u_i has coefficients uniform in $[-2^{k_i-1}, 2^{k_i}-1]$, so on k_i bits. Each of the $\lceil \frac{128}{\log q} \rceil$ polynomials g_i has uniform coefficients mod q ,

except the constant coefficient which is 0, so on $\lceil \log q \rceil$ bits each. Then $n_6 := \sum_{i \in [4]} k_i + (\lceil \log 14\sigma \rceil + 1) + \lceil \frac{128}{\log q} \rceil \cdot \lceil \log q \rceil$ and $\|\mathbf{s}_6\| \leq \beta_6$ for

$$\beta_6 := \sqrt{256 \left(\sum_{i \in [4]} k_i \right) + 256(\lceil \log 14\sigma \rceil + 1) + 255 \cdot \lceil \frac{128}{\log q} \rceil \cdot \lceil \log q \rceil}.$$

The Number of Carries N . We set the number of carries such that the probability that $\|\mathbf{v}_i^{(1)}\|^2 \leq N$ is around 0.99, where $\mathbf{v}_i^{(1)} \in \{-1, 0, 1\}^{256}$ is the vector of high-order values. If we denote by $p := \Pr[\|\mathbf{v}_i^{(1)}[j]\| = 1]$ the probability of a carry, for any entry of index $j \in [256]$, then we can model $\|\mathbf{v}_i^{(1)}\|^2 = \sum_{j \in [256]} (\mathbf{v}_i^{(1)}[j])^2$ as $X \sim \text{Bin}(256, p)$, so we can find N such that its cumulative distribution function, $\Pr[X \leq N] = \sum_{k=0}^N \binom{256}{k} p^k (1-p)^{256-k}$ is close to 0.99. Thus, it suffices to compute the probability p . The masked projection of some witness vector $\mathbf{s}$ at any of the 256 indices looks like $\langle \pi, \mathbf{s} \rangle + u = 2^k v^{(1)} + v^{(0)}$, for some projection row π , uniform mask $u \leftarrow [-2^{k-1}, 2^{k-1} - 1]$, $v^{(1)} \in \{-1, 0, 1\}$ and $v^{(0)} \in [-2^{k-1}, 2^{k-1} - 1]$ and k depends on the norm bound on $\mathbf{s}$. Then a carry happens if the following event E happens: $\langle \pi, \mathbf{s} \rangle + u \geq 2^{k-1}$ or $\langle \pi, \mathbf{s} \rangle + u \leq -2^{k-1} - 1$. Then

$$\begin{aligned} p &= \Pr_{\pi, u}[E] = \sum_{a=-2^k}^{2^k} \Pr_u[E | \langle \pi, \mathbf{s} \rangle = a] \cdot \Pr_{\pi}[\langle \pi, \mathbf{s} \rangle = a] \\ &= \sum_{a=-2^k}^{2^k} p_a \cdot \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{a^2}{2\sigma^2}\right), \end{aligned}$$

where we denoted the first term of the product by p_a and we computed the second term under the assumption that $\langle \pi, \mathbf{s} \rangle \sim N(0, \sigma)$, for $\sigma := \|\mathbf{s}\|/\sqrt{2}$.⁸ The sum above is truncated for $|a| \leq 2^k$, as, for the contrary, $\Pr_{\pi}[\langle \pi, \mathbf{s} \rangle = a] = 0$, by the choice of k . We claim that, for $a = 0$, $p_a = 0$ and else, equals $\frac{|a|}{2^k}$. Then

$$\begin{aligned} p &= 2 \sum_{a=1}^{2^k} \frac{a}{2^k} \cdot \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{a^2}{2\sigma^2}\right) \\ &\leq \frac{1}{2^{k-1} \sigma \sqrt{2\pi}} \int_0^\infty \exp\left(-\frac{a^2}{2\sigma^2}\right) \cdot a \, da \\ &\leq \frac{1}{2^{k-1} \sigma \sqrt{2\pi}} \int_0^\infty \exp(x) \cdot (-\sigma^2) \, dx \\ &\leq \frac{\sigma^2}{2^{k-1} \sigma \sqrt{2\pi}} = \frac{\sigma \sqrt{2}}{2^k \sqrt{\pi}} \approx \frac{1}{9.75 \sqrt{\pi}} \end{aligned}$$

For the first inequality, we use that $\exp(-\frac{x^2}{2\sigma^2})$ is a decreasing positive function, and thus $\exp(-\frac{a^2}{2\sigma^2}) \leq \int_{a-1}^a \exp(-\frac{x^2}{2\sigma^2}) \, dx$.

About the claim on p_a , note that

$$p_a = \Pr_u[a + u \geq 2^{k-1} \text{ or } a + u \leq -2^{k-1} - 1 | \langle \pi, \mathbf{s} \rangle = a].$$

Keep in mind that u takes uniform values over $[-2^{k-1}, 2^{k-1} - 1]$. Then, if $a = 0$, for sure $p_a = 0$. If $0 < a \leq 2^k$, $p_a = \Pr_u[u \geq 2^{k-1} - a] = \frac{a}{2^k}$, whereas, if $-2^k \leq a < 0$, $p_a = \Pr_u[u \leq -2^{k-1} - a - 1] = \frac{|a|}{2^k}$.

Setting $p = \frac{1}{9.75 \sqrt{\pi}}$, we get for $N = 32$ the value $\sum_{k=0}^N \binom{256}{k} p^k (1-p)^{256-k}$ to be close to 0.99.

⁸ It relies on the heuristic assumption of [30] that the entries of any projection row, from Bin_1 , follow a normal distribution of standard deviation $1/\sqrt{2}$.

The Standard Deviations $\sigma, \sigma_1, \sigma_2$. In general, the standard deviation of the Gaussian distribution, used in rejection sampling (Lemma 3) is set as $\gamma \cdot \alpha$, where α is an upper bound of $\|\mathbf{x}\|$, for a vector $\mathbf{x}$ over $\mathcal{R}$ we want to mask, and a repetition constant γ . Recall that by definition of the challenge space $\mathcal{C}$ from Section 2.1, $\|c \cdot \mathbf{x}\| \leq T\alpha$, for any challenge $c \in \mathcal{C}$.

- In the first rejection sampling application, we want to mask $\bar{\Pi}\mathbf{v}_1$, where $\mathbf{v}_1$ is the vector of high-order values. As $\|\mathbf{v}_1\| \leq \sqrt{4 \cdot N}$, we have with high probability $\|\bar{\Pi}\mathbf{v}_1\| \leq \sqrt{337 \cdot 4 \cdot N}$. (Theorem 3) Therefore, we can set $\sigma := \gamma \cdot \sqrt{337 \cdot 4 \cdot N}$.
- In the last rejection sampling application, we want to mask $c(\tilde{\mathbf{s}}, \tilde{\mathbf{v}})$, and $c(\mathbf{r}_s, \mathbf{r}_v)$, for a given challenge $c \in \mathcal{C}$. Note that $\|(\tilde{\mathbf{s}}, \tilde{\mathbf{v}})\|^2 = \sum_{i \in [6]} \|\mathbf{s}_i\|^2 + \|\tilde{\mathbf{v}}_1\|^2 + \|\tilde{\mathbf{v}}_2\|^2 \leq \sum_{i \in [6]} \beta_i^2 + 2 \cdot 4 \cdot N$. Then, we can set $\sigma_1 := \gamma_1 \cdot T \cdot \sqrt{\sum_{i \in [6]} \beta_i^2 + 8N}$. Moreover, $\|(\mathbf{r}_s, \mathbf{r}_v)\|^2 \leq d \cdot (n_s + n_v)$, as these random vectors are made of binary ring elements. Then, we can set $\sigma_2 := \gamma_2 \cdot T \cdot \sqrt{256 \cdot (n_s + n_v)}$.

The parameters γ_1, γ_2 are repetition rate-related constants coming from the rejection sampling algorithms.

The Modulus q for the Next LaBRADOS Iteration. We want the LaBRADOS modulus to coincide with the one of LNP-Lite. To prove correctness of the binary constraint on the vector $\mathbf{x}_{\text{bin}}$, we prove an approximate norm bound of it, $\|\mathbf{x}_{\text{bin}}\| \leq \sqrt{d \cdot n_{\text{bin}}}$, with the slack factor $c_{\text{Lab-new}} = \frac{9.75}{0.74}$. To make sure this constraint does not wraparound mod q , it suffices to set $2\|\mathbf{x}_{\text{bin}}\|^2 \leq q$. This leads to setting

$$2 \cdot d \cdot n_{\text{bin}} \cdot c_{\text{Lab-new}}^2 \leq q.$$

The LNP-Lite Modulus q . One can set the modulus q as indicated by Theorem 2 and as above.

E Security of the Primitives

E.1 Collision Resistant Hash Function

The hash function in Section 4.2 is collision resistant, assuming the intractability of the M-SIS problem with parameters $M = 1$, $N = 16$, $q = 256$, $\beta_\infty = 1$ over $\mathcal{R}$ of degree $d = 512$.

Proof. Let $\mathcal{A}_{\text{CR-hash}}$ be a probabilistic polynomial-time adversary that finds collisions with some advantage ϵ . One constructs a probabilistic polynomial-time adversary $\mathcal{A}_{\text{M-SIS}}$ that solves M-SIS instances with the same advantage as follows:

- $\mathcal{A}_{\text{M-SIS}}$ calls $\mathcal{A}_{\text{CR-hash}}$ to obtain a pair $\mathbf{x} \neq \mathbf{x}' \in \mathcal{R}_2^*$, and aborts if it is not a collision.
- If the inputs $\mathbf{x}, \mathbf{x}'$ have different dimensions, the last block of the padded input is different, $\mathbf{x}_{k-1} \neq \mathbf{x}'_{k'-1}$. Then, there is a collision in the last compression, i.e. $(\mathbf{y}_{k-2}, \mathbf{x}_{k-1}) \neq (\mathbf{y}'_{k'-2}, \mathbf{x}'_{k'-1}) \in \mathcal{R}_2^{16}$ with $\mathbf{A}(\mathbf{y}_{k-2}, \mathbf{x}_{k-1}) = \mathbf{A}(\mathbf{y}'_{k'-2}, \mathbf{x}'_{k'-1})$. $\mathcal{A}_{\text{M-SIS}}$ returns the difference $(\mathbf{y}_{k-2}, \mathbf{x}_{k-1}) - (\mathbf{y}'_{k'-2}, \mathbf{x}'_{k'-1})$, which is a non-zero vector $\mathbf{v}$ with coefficients in $\{-1, 0, 1\}$ satisfying $\mathbf{A}\mathbf{v} = 0 \bmod 256$.
- Else, since $\mathbf{x} \neq \mathbf{x}'$, the number of blocks of the padded inputs is equal and there exists $\mathbf{x}_i \neq \mathbf{x}'_i$. It follows that either $(\mathbf{y}_{i-1}, \mathbf{x}_i) \neq (\mathbf{y}'_{i-1}, \mathbf{x}'_i)$ and $\mathbf{A}(\mathbf{y}_{i-1}, \mathbf{x}_i) = \mathbf{A}(\mathbf{y}'_{i-1}, \mathbf{x}'_i)$ or $\mathbf{y}_i \neq \mathbf{y}'_i$, hence $(\mathbf{y}_i, \mathbf{x}_{i+1}) \neq (\mathbf{y}'_i, \mathbf{x}'_{i+1})$. In the first case, $\mathcal{A}_{\text{M-SIS}}$ returns the difference $(\mathbf{y}_{i-1}, \mathbf{x}_i) - (\mathbf{y}'_{i-1}, \mathbf{x}'_i)$. In the second case, it proceeds until it finds an iteration in the compression function and returns the difference.

We estimate the concrete hardness of the SIS problem with d rows, $16d$ columns, ℓ_2 norm bound $\beta = \sqrt{16d}$ as at least 256-bits secure, by the Lattice Estimator [4], commit 6019056.

E.2 Pseudorandom Generator

The function in Section 4.1 is a pseudorandom generator, assuming the hardness of some Module-LWR problem.

Proof. Let $\mathcal{A}_{\text{PRG}}$ be a probabilistic polynomial-time algorithm that distinguishes the PRG output from a uniformly random one with advantage ϵ_{PRG} , i.e.

$$\begin{aligned} & \left| \Pr[\mathcal{A}_{\text{PRG}}(\mathbf{A}, \mathbf{s}) = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_{256}^{1 \times 2}, s_0^{(6)}, s_0^{(7)} \leftarrow \mathcal{R}_2, \mathbf{s} = \text{PRG}(s_0^{(6)}, s_0^{(7)})] \right. \\ & \quad \left. - \Pr[\mathcal{A}_{\text{PRG}}(\mathbf{A}, \mathbf{s}) = 1 \mid \mathbf{A} \leftarrow \mathcal{R}_{256}^{1 \times 2}, \mathbf{s} \leftarrow \mathcal{R}_2^{4k+2}] \right| = \epsilon_{\text{PRG}}. \end{aligned}$$

One constructs a probabilistic polynomial-time algorithm $\mathcal{A}_{\text{M-LWR}}$ that distinguishes M-LWR samples from uniformly random ones with advantage $\epsilon_{\text{MLWR}} = \epsilon_{\text{PRG}}/k$ as follows.

- Receive as input a pair $(\mathbf{A}, t)$ where $\mathbf{A} \leftarrow \mathcal{R}_{256}^{1 \times 2}$ and $t \in \mathcal{R}_{64}$.
- Sample $i \leftarrow [k]$ and let $\mathbf{s}_i$ be the decomposition of t , $t = \mathbf{G}_2 \mathbf{s}_i$.
- Sample $s_j^{(2)}, \dots, s_j^{(5)} \leftarrow \mathcal{R}_2$ for $j \in [i-1]$.
- Compute $\lceil \mathbf{A}(s_j^{(6)}, s_j^{(7)}) \rceil_{256 \rightarrow 64} = \mathbf{G}_2 \mathbf{s}_{j+1}$ for $j \in [i, k-1]$.
- Return $\mathcal{A}_{\text{PRG}}(\mathbf{A}, \mathbf{s})$.

For $i = 0, \dots, k$, let the *hybrid* H_i describe the distribution

$$\begin{aligned} H_i = \{(\mathbf{A}, \mathbf{s}) : & \mathbf{A} \leftarrow \mathcal{R}_{256}^{1 \times 2}, s_j^{(2)}, \dots, s_j^{(5)} \leftarrow \mathcal{R}_2 \text{ for } j \in [i], \\ & s_i^{(6)}, s_i^{(7)} \leftarrow \mathcal{R}_2, \lceil \mathbf{A}(s_j^{(6)}, s_j^{(7)}) \rceil_{256 \rightarrow 64} = \mathbf{G}_2 \mathbf{s}_{j+1} \text{ for } j \in [i, k-1]\}. \end{aligned}$$

Note that H_0 describes the evaluation of the pseudorandom generator on a random input $s_0^{(6)}, s_0^{(7)} \leftarrow \mathcal{R}_2$, while H_k presents a random binary output. Then, ϵ_{PRG} can be written as

$$\left| \Pr[\mathcal{A}_{\text{PRG}}(H_0) = 1] - \Pr[\mathcal{A}_{\text{PRG}}(H_k) = 1] \right|.$$

By the construction of $\mathcal{A}_{\text{M-LWR}}$ and the union bound, we have $\Pr[\mathcal{A}_{\text{M-LWR}}(\mathbf{A}, t) = 1 \mid t = \lceil \mathbf{A}\mathbf{s} \rceil_{256 \rightarrow 64}] = \frac{1}{k} \sum_{i=1}^k \Pr[\mathcal{A}(H_{i-1}) = 1]$ and $\Pr[\mathcal{A}_{\text{M-LWR}}(\mathbf{A}, t) = 1 \mid t \leftarrow \mathcal{R}_{64}] = \frac{1}{k} \sum_{i=1}^k \Pr[\mathcal{A}(H_i) = 1]$, then

$$\begin{aligned} \epsilon_{\text{M-LWR}} &= \frac{1}{k} \left| \sum_{i=1}^k \Pr[\mathcal{A}_{\text{PRG}}(H_{i-1}) = 1] - \Pr[\mathcal{A}_{\text{PRG}}(H_i) = 1] \right| \\ &= \frac{1}{k} \left| \Pr[\mathcal{A}_{\text{PRG}}(H_0) = 1] - \Pr[\mathcal{A}_{\text{PRG}}(H_k) = 1] \right| = \frac{1}{k} \epsilon_{\text{PRG}}. \end{aligned}$$

To estimate the concrete hardness of the Module-LWR problem with parameters $q = 256$, $p = 64$, $m' = 1$, $n' = 2$, $\chi = \mathcal{U}(\mathcal{R}_2)$, we model the LWR problem as an LWE problem, with d rows, $2 \cdot d$ columns, modulus q , secret distribution as χ , and error distribution as Gaussian of standard deviation $\sqrt{((q/p)^2 - 1)/12}$. ([2, 14]). According to the Lattice Estimator, [4], commit 6019056, this instance costs at least 2^{300} ring operations.

E.3 Output Transformation

The modulus transformation in Section 4.1 is, intuitively, an expander that converts a binary input into ring elements. For simplicity, we chose the output entropy of the PRG to be the same as the size of the ring and we perform the mapping via a matrix-vector multiplication where the matrix consists of random ring elements from the target ring. If the binary input to this multiplication is uniformly-random, then one can view the output distribution as a Ring-LWE instance. Because the entropy of the input is as large as the output, this in fact the hardest LWE instance for the ring. The security of this transformation is therefore much larger than the security of the PRG (roughly 7000 bits), and we include the formal analysis below merely for completeness.

If we treat the input to this expander as random, then the output is indistinguishable from a random element, assuming the hardness of some Module-LWE problem.

Proof. The proof is analogous to the previous one. From the adversary $\mathcal{A}_{\text{mod}}$ that distinguishes the output of the function from a random one with advantage ϵ_{mod} , one constructs a Module-LWE adversary $\mathcal{A}_{\text{M-LWE}}$ with advantage ϵ_{mod}/l as follows

- sample $i \leftarrow [l]$ and set $t_i = t$,
- sample $t_j \leftarrow \mathcal{R}_q$ for $j \in [0, i-1]$,
- compute $t_j = \mathbf{B}\tilde{\mathbf{s}}_j$ with $\tilde{\mathbf{s}}_j \leftarrow \mathcal{R}_2^\ell$ for $j \in [i+1, l-1]$.
- return $\mathcal{A}_{\text{mod}}(\mathbf{B}, \mathbf{t})$ with $\mathbf{t} = (t_0, t_1, \dots)^T$.

The hybrids are defined, for $i \in [0, l]$ as

$$H_i = \{(\mathbf{B}, \mathbf{t}) : t_j \leftarrow \mathcal{R}_q \text{ for } j \in [0, i-1], \tilde{\mathbf{s}}_j \leftarrow \mathcal{R}_2^\ell \\ t_j = \mathbf{B}\tilde{\mathbf{s}}_j \bmod q \text{ for } j \in [i, l-1]\}.$$

The concrete hardness of the LWE problem with d rows, $d \cdot \lceil \log(q) \rceil$ columns, secret and error distributions uniform binary, for the Falcon modulus $q = 12289$, is estimated to be at least 7000-bits secure by the Lattice Estimator. [4].